Verification Continuum™

VDK for NXP S32K56x VP IP User Guide

Version R6.0.0
October 2025

Copyright Notice and Proprietary Information

© 2025 Synopsys, Inc. ALL RIGHTS RESERVED.

This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Licensed Products communicate with Synopsys servers for the purpose of providing software updates, detecting software piracy and verifying that customers are using Licensed Products in conformity with the applicable License Key for such Licensed Products. Synopsys will use information gathered in connection with this process to deliver software updates and pursue software pirates and infringers.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at
https://www.synopsys.com/company/legal/trademarks-brands.html
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

http://www.synopsys.com

Contents

Preface 12
About This Manual . . . 13
Documentation Conventions . . . 14
Terminology . . . 16
Customer Support . . . 21
Synopsys Statement on Inclusivity and Diversity . . . 22

1 Analog-to-Digital Converter (SARADC) 23
1.1 Introduction . . . 23
1.2 Supported Features . . . 23
1.3 Features Not Supported . . . 24
1.4 Assumptions . . . 24
1.5 SCML Parameters . . . 24
1.6 SCML Commands . . . 26

2 AVPLL (PLL) 27
2.1 Introduction . . . 27
2.2 Supported Features . . . 27
2.3 Features Not Supported . . . 27
2.4 Assumptions . . . 28
2.5 SCML COMMANDS . . . 28

3 Body Cross-triggering Unit (BCTU) 29
3.1 Introduction . . . 29
3.2 Supported Features . . . 29
3.3 Features Not Supported . . . 29
3.4 Assumptions . . . 30

4 Central Self-Test Control Unit (CSTCU) 31
4.1 Features Not Supported . . . 31
4.2 SCML Commands . . . 31

5 Clock Generation Module (MC_CGM) 33
5.1 Introduction . . . 33
5.2 Supported Features . . . 33
5.3 Features Not Supported . . . 33
5.4 Assumption . . . 34

6 Clock Monitoring Units (CMU) 35

6.1 Introduction . . . 35
6.2 Features Not Supported . . . 35

7 CPE_GPR **36**
7.1 Features Not Supported . . . 36

8 Crossbar Integrity Checker (XBIC) **37**
8.1 Introduction . . . 37
8.2 Supported Features . . . 37
8.3 Features Not Supported . . . 37
8.4 Assumptions . . . 37
8.5 SCML Parameters . . . 37
8.6 SCML Commands . . . 38

9 Cyclic Redundancy Check (CRC) **39**
9.1 Introduction . . . 39
9.2 Supported Features . . . 39

10 De-serial Serial Peripheral Interface (DSPI) **40**
10.1 Introduction . . . 40
10.2 Supported Features . . . 40
10.3 Features Not Supported . . . 41
10.4 Assumptions . . . 42

11 Direct Memory Access Multiplexer (DMAMUX) **43**
11.1 Introduction . . . 43
11.2 Supported Features . . . 43

12 DMA Cyclic Redundancy Check(DMA_CRC) **44**
12.1 Introduction . . . 44
12.2 Supported Features . . . 44
12.3 Features Not Supported . . . 45
12.4 Assumptions . . . 45

13 DSP_GPR **46**
13.1 Supported Features . . . 46
13.2 Features Not Supported . . . 46

14 DSP-V Subsystem (DSP-V) **47**
14.1 Introduction . . . 47
14.2 Supported Features . . . 47
14.3 Features Not Supported . . . 48

15 ENET_PHY_WRAPPER (SERDES) **49**
15.1 Features Not Supported . . . 49

16 Enhanced Direct Memory Access (eDMA) **50**
16.1 Introduction . . . 50
16.2 Supported Features . . . 50

16.3 Features Not Supported . . . 51
16.4 Assumptions . . . 51
16.5 SCML Objects . . . 51
16.6 SCML Commands . . . 52

17 Enhanced Direct Memory Access (eDMA5) 53
17.1 Introduction . . . 53
17.2 Supported Features . . . 53
17.3 Features Not Supported . . . 54
17.4 Assumptions . . . 54
17.5 SCML Objects . . . 55
17.6 SCML Commands . . . 55

18 Enhanced Direct Memory Access Cyclic Redundancy Check (eDMA3_CRC) 56
18.1 Introduction . . . 56
18.2 Supported Features . . . 56
18.3 Features Not Supported . . . 57
18.4 Assumptions . . . 57

19 Error Injection Module (EIM) 58
19.1 Introduction . . . 58
19.2 Supported Features . . . 58
19.3 Features Not Supported . . . 58
19.4 Assumptions . . . 58
19.5 SCML Parameters . . . 58

20 Error Reporting Module (ERM) 60
20.1 Supported Features . . . 60
20.2 Assumptions . . . 60
20.3 SCML Commands . . . 61

21 Extended Resource Domain Controller (XRDC) 62
21.1 Introduction . . . 62
21.2 Supported Features . . . 62
21.3 Features Not Supported . . . 63

22 External Serial Peripheral Interface (XSPI) 64
22.1 Supported Features . . . 64
22.2 Features Not Supported . . . 64
22.3 Assumptions and Dependencies . . . 65
22.4 SCML Commands . . . 65

23 Fast Crystal Oscillator (FXOSC) 66
23.1 Introduction . . . 66
23.2 Supported Features . . . 66
23.3 Features Not Supported . . . 66
23.4 Assumptions . . . 66

24 Flex Timer Module (FTM) 67

24.1 Introduction . . . 67
24.2 Supported Features . . . 67
24.3 Features Not Supported . . . 68
24.4 Assumptions . . . 68

25 Flexible Data Rate CAN (FlexCAN) 69
25.1 Introduction . . . 69
25.2 Supported Features . . . 69
25.3 Features Not Supported . . . 70
25.4 Assumptions . . . 71

26 Flexible I/O (FlexIO) 72
26.1 Introduction . . . 72
26.2 Supported Features . . . 72
26.3 Features Not Supported . . . 72
26.4 Assumptions . . . 73

27 Free-Running Oscillator Clock Generator (FRO) 74
27.1 Introduction . . . 74
27.2 Supported Features . . . 74
27.3 Features Not Supported . . . 74
27.4 SCML Commands . . . 74

28 General Purpose Registers 0 (GPR0) 76
28.1 Introduction . . . 76
28.2 Supported Features . . . 76
28.3 Features Not Supported . . . 76

29 General Purpose Registers 1 (GPR1) 77
29.1 Introduction . . . 77
29.2 Supported Features . . . 77
29.3 Features Not Supported . . . 77

30 GPR_LPE_0 78
30.1 Supported Features . . . 78
30.2 Features Not Supported . . . 78

31 GPR_LPE_1 79
31.1 Features Not Supported . . . 79

32 Interrupt Controller (INTC) 80
32.1 Introduction . . . 80
32.2 Supported Features . . . 80
32.3 Features Not Supported . . . 80
32.4 Assumptions . . . 81

33 Local Self-Test Control Unit (LSTCU) 82
33.1 Features Not Supported . . . 82
33.2 SCML Commands . . . 82

34 Logic Control Unit (LCU) 84
34.1 Introduction . . . 84
34.2 Supported Features . . . 84
34.3 Assumptions . . . 84

35 Low Power Comparator (LPCMP) 85
35.1 Supported Features . . . 85
35.2 Features Not Supported . . . 85
35.3 Assumptions and Dependencies . . . 85

36 Low Power Inter-Integrated Circuit (LPI2C) 86
36.1 Introduction . . . 86
36.2 Supported Features . . . 86
36.3 Features Not Supported . . . 87
36.4 Assumptions . . . 87
36.5 SCML Parameters . . . 87
36.6 SCML Commands . . . 87

37 Low Power Serial Peripheral Interface (LPSPI) 89
37.1 Introduction . . . 89
37.2 Supported Features . . . 89
37.3 Features Not Supported . . . 90
37.4 Assumptions . . . 90

38 Low-Power Universal Asynchronous Receiver/Transmitter (LPUART) 91
38.1 Introduction . . . 91
38.2 Supported Features . . . 91
38.3 Features Not Supported . . . 92
38.4 Assumptions . . . 92

39 Message Interrupt (MSGINTR) 93
39.1 Introduction . . . 93
39.2 Supported Features . . . 93
39.3 Assumptions . . . 93

40 Message Receive Unit (MRU) 94
40.1 Introduction . . . 94
40.2 Supported Features . . . 94

41 Messaging Unit (MU) 95
41.1 Introduction . . . 95
41.2 Supported Features . . . 95
41.3 Features Not Supported . . . 96

42 Miscellaneous Control Module (MCM) 97
42.1 Introduction . . . 97
42.2 Supported Features . . . 97
42.3 Features Not Supported . . . 97

43 Miscellaneous System Control Module (MS_CM) 98
43.1 Introduction . . . 98
43.2 Supported Features . . . 98
43.3 Features Not Supported . . . 98
43.4 Assumptions . . . 98

44 Mode Entry Module (MC_ME) 100
44.1 Introduction . . . 100
44.2 Supported Features . . . 100
44.3 Assumptions . . . 100

45 MRAM Controller (MRAMC) 102
45.1 Introduction . . . 102
45.2 Supported Features . . . 102
45.3 Features Not Supported . . . 102
45.4 Assumptions . . . 103
45.5 SCML Commands . . . 103

46 Periodic Interrupt Timer (PIT) 104
46.1 Introduction . . . 104
46.2 Supported Features . . . 104
46.3 Features Not Supported . . . 104
46.4 Assumptions . . . 104

47 Power Control Unit (MC_PCU) 105
47.1 Introduction . . . 105
47.2 Features Not Supported . . . 105

48 Power Management Controller (PMC) 106
48.1 Introduction . . . 106
48.2 Features Not Supported . . . 106
48.3 Assumptions . . . 107

49 Power-on Reset Generator (POR_GEN) 108
49.1 Introduction . . . 108
49.2 Supported Features . . . 108

50 Real Time Clock (RTC) 109
50.1 Introduction . . . 109
50.2 Supported Features . . . 109
50.3 Features Not Supported . . . 110
50.4 Assumptions . . . 110

51 Reset Domain Controller (RDC) 111
51.1 Introduction . . . 111
51.2 Supported Features . . . 111
51.3 Features Not Supported . . . 111
51.4 Assumptions . . . 111

52 Reset Generation Module (MC_RGM) 112
52.1 Introduction . . . 112
52.2 Supported Features . . . 112
52.3 Features Not Supported . . . 112
52.4 Assumption . . . 113

53 RX Lookup Accelerator (RXLUT) 114
53.1 Introduction . . . 114
53.2 Supported Features . . . 114
53.3 Features Not Supported . . . 115

54 Semaphores2 (SEMA42) 116
54.1 Introduction . . . 116
54.2 Supported Features . . . 116
54.3 SCML Parameters . . . 117

55 Sinc Filter (SINC) 118
55.1 Introduction . . . 118
55.2 Supported Features . . . 118
55.3 Features Not Supported . . . 118
55.4 SCML COMMANDS . . . 119

56 Slow Crystal Oscillator Digital Controller (SXOSC) 120
56.1 Introduction . . . 120
56.2 Supported Features . . . 120
56.3 Features Not Supported . . . 120
56.4 Assumptions . . . 120

57 Software Watchdog Timer (SWT) 121
57.1 Introduction . . . 121
57.2 Supported Features . . . 121
57.3 Features Not Supported . . . 121
57.4 Assumptions . . . 122

58 Synchronous Audio Interface (SAI) 123
58.1 Introduction . . . 123
58.2 Features Not Supported . . . 123
58.3 Assumptions . . . 124
58.4 SCML Commands . . . 124

59 System Integration Unit Lite2 (SIUL2) 125
59.1 Introduction . . . 125
59.2 Supported Features . . . 125
59.3 Features Not Supported . . . 125
59.4 Assumptions . . . 125

60 System RAM (SRAMC) 126
60.1 Introduction . . . 126
60.2 Supported Features . . . 126

60.3 Features Not Supported . . . 126
60.4 Assumptions . . . 127
60.5 SCML Parameters . . . 127
60.6 SCML Commands . . . 127

61 System Timer Module (STM) 129
61.1 Introduction . . . 129
61.2 Supported Features . . . 129
61.3 Features Not Supported . . . 129

62 Temperatire Sensor Unit (TEMPSENSE) 130
62.1 Introduction . . . 130
62.2 Supported Features . . . 130
62.3 Features Not Supported . . . 130

63 Trigger MUX (TRGMUX) 131
63.1 Introduction . . . 131
63.2 Supported Features . . . 131
63.3 Assumptions . . . 131
63.4 SCML Parameters . . . 131
63.5 SCML Commands . . . 132

64 Ultra Secured Digital Host Controller (uSDHC) 133
64.1 Introduction . . . 133
64.2 Features Not Supported . . . 134

65 Virtual Wrapper (Virt_Wrapper) 135
65.1 Introduction . . . 135
65.2 Supported Features . . . 135
65.3 Assumptions . . . 135

66 Virtualized Fault Collection and Control Unit (VFCCU) 136
66.1 Introduction . . . 136
66.2 Supported Features . . . 136
66.3 Features Not Supported . . . 136
66.4 Assumptions . . . 136
66.5 SCML Commands . . . 137

67 Wakeup Unit (WKPU) 138
67.1 Introduction . . . 138
67.2 Supported Features . . . 138
67.3 Features Not Supported . . . 138

68 CANXL 139
68.1 Introduction . . . 139
68.2 Supported Features . . . 139
68.3 Features Not Supported . . . 139
68.4 Assumptions . . . 140
68.5 SCML Commands . . . 140

69 NETC **141**
69.1 Introduction 141
69.2 Supported Features 141
69.3 Features Not Supported 142
69.4 Assumptions 143
69.5 SCML Commands 143

70 NPU **144**
70.1 Introduction 144
70.2 Supported Features 144
70.3 Features Not Supported 144
70.4 Assumptions 144
70.5 SCML Commands 144

71 S32K5 HIERARCHICAL PATHS **145**
71.1 Hierarchical Paths 145

Preface

This preface includes the following sections:

- About This Manual
- Documentation Conventions
- Terminology
- Customer Support
- Synopsys Statement on Inclusivity and Diversity

About This Manual

This document describes the peripheral models of VDK for NXP S32K56x VP IP User Guide.

Documentation Conventions

This section lists and explains the documentation conventions used throughout this manual.

Convention	Description and Examples
italic	Is used in running text for: • GUI elements. For example: The *Enumeration* field contains a space-separated list of values. • New terms. For example: A *protocol library* is a collection of protocol definitions. • Web sites. For example: For more information, see http://www.eclipse.org. • E-mail addresses. For example: Please contact customer support via e-mail at *vp_support@synopsys.com*. • Manual names. For example: The preface of the *Analysis Manual* describes:

`courier`	Is used for:
	• Code text. For example:
	`list_library_configurations`
	`myConfig`
	In this example, `myConfig` is used.
	• System messages. For example:
	`JVM not found.`
	• Text you must type literally. For example:
	At the prompt, type `go`.
	• Names (of environment variables, commands, utilities, prompts, paths, macros, and so on). For example:
	The `build-options` command sets build parameters.

`courieritalic`	Indicates variables. For example: *`scope`* specifies a module, a channel, or a refined port.
bold	Serves to draw your attention to the text in question. For example: `coreId = cwrSAGetCoreId("ARC_HS38");`

`[ ]`	Square brackets enclose optional items. For example:
	`clean [-pch]`
	If you must type a square bracket as part of the syntax, it is enclosed in single quotes. For example:
	`'['--use-vector']'`

`{ }`	Braces enclose a list from which you must choose one or more items. For example:
	`add {signalPattern | portPattern}` *ID*
	If you must type a brace as part of the syntax, it is enclosed in single quotes. For example:
	`DECLARE '{'`
	`Item1`
	`Item2`
	`'}'`

`|`	A vertical bar separates items in a list of choices. For example: `autoflush {on | off}`
>	A right angle bracket separates menu commands. For example: The *Project* > *Update System Library* menu command is available.

...	A horizontal ellipsis in syntax indicates that the preceding expression may have zero, one, or more occurrences. For example:
	`build-options -option optionArgs ...`
	A horizontal ellipsis in examples and system messages indicates material that has been omitted. For example:
	`::scsh> dtrace add top1.signal_* $t1`
	`::scsh> dtrace add top1.clk_* $t1`
	`...`
	`::scsh> dtrace flush *`

Terminology

Terminology	Description
ADC	Analog to Digital Converter
ADCQ	Queued SARADC Digital Subsystem
AHB	Advanced High-Performance Bus
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
API	Autonomous Periodic Interrupt
APLL	Audio PLL
AUX	Auxillary
AUXQ	Auxillary Queue
AXI	Advanced Extensible Interface
BIST	Built-In Self-Test
CAN	Controller Area Network
CEM	Collective Error Manager
CGM	Clock Generator Module
CMDQ	Command Queue
CMU	Clock Monitoring Unit
CPHA	Clock Phase Bit
CPOL	Clock Polarity Bit
CPU	Central Processing Unit
CQE	Command Queueing Engine
CQRS	Command Queue Register Set
CRC	Cyclic Redundancy Check
CRS	Common Register Set
CTU	Cross Triggering Unit
DATAQ	Data Queue
DCP	Design Checkpoint
DDR	Double Data Rate

DMA	Direct Memory Access Controller
DMAMUX	Direct Memory Access Controller Multiplexer
DMU	Data Cache Miss Unit
DRAM	Dynamic Random Access Memory
DRLL	Digital Ratio Locked Loop
DTC	Digital Tool Chain
ECC	Error Correction Code
ECU	Electronic Control Unit
eDMA	Enhanced Direct Memory Access
eMIOS	Enhanced Modular IO Subsystem
eMMC	Embedded Multimedia Card
EEPROM	Electrically Erasable Programmable Read-Only Memory
ERR	Equal Resistance Routing
FCCU	Fault Collection and Control Unit
FIFO	First In First Out
FSM	Finite State Machine
FSRC	Fast Sample Rate Converter
FT	Fast Timed
FTP	File Transfer Protocol
GCP	Global Clock Prescaler
GIC	Generic Interrupt Controller
GN	Ground
GPIO	General Purpose Input Output
GPV	Global Programmers View
GST	Global System Timer
GSTRI	Global System Time Read Interface
GTM	Generic Timer Module
GUI	Graphical User Interface
HCI	Host Controller Interface

HRS	Host Register Set
HTTP	Hypertext Markup Language
IBCM	Interrupt Broadcaster
I2C	Inter-Integrated Circuit
ID	Identifier
IP	Intellectual Property. Commonly used term for a re-usable hardware component in source code or library form. Also used to refer to virtual models of hardware components.
IPS	Intrusion Prevention System
IRCOSC	Internal RC Oscillator
I2S	Integrated Interchip Sound
JTAG	Joint Test Action Group
LCD	Liquid Crytsal Display
LIN	Local Interconnect Network
LINFlexD	Local Interconnect Network Controller
LPDDR	Low-Power Double Data Rate
LPR	Low Priority Read
LPRC	Low Power RC Oscillator
LSB	Least Significant Bit
LSBFE	Least Significant Bit First Enable
LT	Loosely Timed
MAC	Media Access Control
MB	Message Buffer
MCR	Match Control Register
MC_RGM	Reset Generation Module
MMC	Multi-Media Card
MSB	Most Significant Bit
MSP	Multi-Channel Serial Port
MTFE	Modified Transfer Format
MUX	Multiplexer

NACK	Negative Acknowledgement
NMI	Non Maskable Interrupt
NOC	Network on Chip
NSC	Nano Second Counter
NVIC	Nested Vectored Interrupt Controller
NTP	Network Time Protocol
OSCAR	Operational Safety Checker and Reporter
PCR	Protocol Configuration Register
PCT	Platform Creator
PCS	Physical Coding Sublayer
PERFM	Performance-Monitoring
PIT	Periodic Interrupt Timer
PLL	Phase Lock Loop
PMC	Power Management Controller
PMM	Path Margin Monitor
PMU	Power Management Unit
P2P	Port to Port
POR	Power On Reset
PPI	PHY Protocol Interface
PRAM	Platform Random Access Memory
PSI5	Peripheral Sensor Interface 5
PVTC	PVT Controller
RAM	Random Access Memory
RCC	Redundancy Control Checker
RDI	Raw Die to Die Interface
RFS	Receive Frame Synchronization
RTC	Real Time Clock
RTCVAL	Real Time Clock
RTL	Register Transfer Level

SARADC	Successive Approximation Register Analog-to-Digital Convertor
SCL	Serial Clock
SCML	SystemC Modeling Library
SD	Secure Digital
SDA	Serial Data
SDADC	Sigma Delta Analog-to-Digital Converter
SDIO	Secure Digital Input Output
SDR	Single Data Rate
SENT	Single Edge Nibble Transmission
SID	Stream Identifier
SIDG	StreamID Generator
SIMD	Streaming SIMD Extensions
SIUL2	System Integration Unit Lite 2
SMC	Simultaneous Multi Corner
SME	Secure Memory Encryption
SoC	System on Chip
SPI	Serial Peripheral Interface
SPIQ	Queued Serial Peripheral Interface
SRAM	Static Random Access Memory
SRM	Security Risk Management
SRS	Slot Register Set
SRX	SENT Receiver
SSID	Sub Stream Identifier
SWT	Software Watchdog Timer
TBU	Translation Buffer Unit
TDI	TEE Device Interface
TDLM	Tamper Detect Logic Module
TDM	Time Division Multiplexed
TFS	Transmit Frame Synchronization

TLM	Transcation-Level Modeling
TxBCIE	Tx Buffer Cancellation Finished Interrupt Enable
TxBCF	Tx Buffer Cancellation Finished
TxBRP	Tx Buffer Request Pending
TxBTIE	Tx Buffer Transmission Interrupt Enable
TxBTO	Tx Buffer Transmission Occurred
UAC	User Account Control
UART	Universal Asynchronous Receiver-Transmitter
UC	Unified Communications
UNE	Universal Network Engine
VDK	Virtual Development Kit
VDKC	Virtual Development Kit Creator
VMID	Virtual Machine Identifier
VP	Virtual Prototyping
VPX	Virtual Prototyping Explorer
XGMII	10-Gigabit Media-Independent Interface
XOSC	External Crystal Oscillator

Customer Support

For technical support (regarding license keys, IP downloads, Host ID, Project ID, documentation or general support), contact the Support Center with a description of your question and supplying the debug information, using one of the following methods:

- Go to https://solvnetplus.synopsys.com and sign-in with your Synopsys SolvNetPlus credentials. Select *Cases* from the menu bar, and select *Create a New Case*. Provide the requested information, including:
  - **Product L1:** Virtual Prototyping.
  - **Product L2:** Select the product type that closest matches yours.
  - **Case Type:** Select the case type from the drop-down menu.
  - **Case Severity:** Select the case severity from the drop-down menu.
  - **Subject:** Provide a brief summary of the issue or list the error message you have encountered.
  - **Description:** For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood.
    After creating the case, attach the debug files you have created, if any.
- Or, send an e-mail message to *vp_support@synopsys.com*. (your email will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):

  - Include the Product name, Sub Product name, and Tool Version number in your email; so that it can be routed correctly. For this information, see `$SNPS_VP_HOME/common/etc/release_name.txt` in your installation.
  - Include exact Patch-level detail of the Tool Version also in your email. To view the list of patches installed for the tool, execute the command: `$SNPS_VP_HOME/common/bin/patch_history.sh`.
  - For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood.
  - Attach any debug files you created in the previous step.
- Or, telephone your local support center:
  - North America:

    Call 1-800-245-8005 from 7:00 AM to 5:30 PM Pacific time, Monday through Friday.
  - All other countries:

    https://www.synopsys.com/support/global-support-centers.html

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Chapter 1
Analog-to-Digital Converter (SARADC)

1.1 Introduction

The ADC module has configurable number of input channels for each group namely:

- Group0 (channel 0 to channel 31)
- Group32 (channel 32 to channel 63)
- Group64 (channel 64 to channel 95)

SARADC produces a digital value from an input analog voltage using the Successive Approximation Register (SAR) algorithm which provides fast and accurate conversion. It features various methods to trigger a conversion and offers flexibility in the selection of input channels. SARADC is useful for a wide range of applications.

1.2 Supported Features

The SARADC module implements the following features:

- Three modes of trigger conversion are supported:
  - Normal conversion with one shot and scan operation mode.
  - Injected conversion with only one-shot mode.
  - CTU triggered mode with single conversion per trigger only.
- Normal/injected conversion supports software as well as external hardware trigger.
- All three modes of conversion: Single Ended mode, Differential mode, Dual parallel mode.
- Abort feature in normal/injected mode for both single or a chain of conversions.
- Configurable number of input channels.
- Independent data registers for each channel with information about mode of conversion, data valid, overwritten status along with data.
- Programmable analog watchdogs configurable for different channels.
- DMA transfer of conversion data with programmable DMA enable for each channel.
- Individual interrupt flags for the following conditions:
  - End of conversion of single channel for normal and injected mode.
  - End of conversion chain for both normal and injected mode.
  - End of conversion in CTU mode.
  - Watchdog thresholds violation.

- Programmable Clock pre-scalar for SAR (bus clock or bus clock divided by 2).
- Software initiated calibration with help of SCML command.
- Self-test feature.
- Fault indication to Fault Control Unit.
- Power Down mode to put ADC in power down state.

1.3 Features Not Supported

The following features are not implemented in SARADC module:

- Auto clock off feature: software interface is modelled but functionality has not been implemented.
- MCR.NSTART bit can be reset by software to allow go into Power down mode while in Scan mode.
- With CALSTAT register, individual step status for calibration is not implemented. Either all steps passes, or all fails.
- Below bitfields are abstracted.
- CAL2.ENX bit is abstracted.
- CTR1.TSENSOR_SEL.
- AMSIO.HSEN.
- ANAMUX is not implemented.

1.4 Assumptions

The following assumption are taken while modelling the SARADC module:

- Support for REF_RANGE interrupt is provided via an SCML command.
- If ADC is in CTU conversion mode, ABORT and ABORTCHAIN doesn't impact the conversion operation.
- For self-testing, a default value of converted data is supplied through configuration parameter, and users would be allowed to overwrite it, at run-time, using a scml_command.
- If the configuration of registers (mode changes/sampling/conversion time changes) is done while a conversion is ongoing, the changes are reflected at the next start of conversion.
- p_cf_out and p_ncf_out are cleared at the start of normal conversions if they are asserted during previous conversion.
- If MCR[TRGEN] bit is set and MCR[XSTRTEN] bit is set, then external normal trigger waits for positive edge of sync pulse trigger to start conversion, and if MCR[TRGEN] bit is set and MCR[XSTRTEN] bit is zero, then external normal trigger starts conversion immediately.
- Every functionality is same for both SARADC_P and SARADC_M.

1.5 SCML Parameters

The following table describes the SCML parameters used by the SARADC module.

Name	Default Value	Description
RESOLUTION	16	Set as per bit size of CDR data conversion.
SELF_TEST_INT_DATA	0x0	Used during self-testing, for every algorithm (S, O and C), until the value is overwritten by a new value supplied via scml::command.
SELF_TEST_FRAC_DATA	0x0	Used during self-testing for algorithm S, step 1, until this value is overwritten by a new value supplied via scml command.
INSTANCE	"CC_0"	Instance name of the SARADC in platform.
NUM_GRP0CH	32	ADC channels from ADC0 to ADC31 - aka precision channels.
NUM_GRP32CH	22	ADC channels from ADC32 to ADC63 - aka Internalchannels.
NUM_GRP64CH	32	ADC channels from ADC64 to ADC95 - aka External channels.
NUM_CHAN_REG	((NUM_GRP64CH>0)?3:((NUM_GRP32CH>0)?2: ((NUM_GRP0CH>0)?1 : 0)))	Number of channel dependent registers, that is, register whose count is dependent on Number of channels.
NUM_WATCHDOG	4	Number of watchdogs
STAW0R_rstVal	0x57EB451C	
NUM_CHANNELS	((NUM_GRP64CH>0)? (NUM_GRP64CH+64):((NUM_GRP32CH>0)? (NUM_-GRP32CH+32): ((NUM_-GRP0CH>0)? (NUM_-GRP0CH) : 0)))	Total number of channels, auto calculated.
CTU_EN	1	Determines if CTU Interface is enabled or not.
CTUMODE_EN	2	Determines what mode is supported with CTU; only valid if CTU_EN is 1.
0 - CTU always in trigger mode.		

1 - CTU always in control mode.		
2 - CTU mode is configurable with MCR.CTU_MODE field.		
STCR1_rstVal	0x0	
STAW2R_rstVal	0x782F4CA9	
STAW3R_rstVal		
STAW4o5R_rstVal		
MCR_rstVal	0x00000001	
STAW1R_rstVal	0x00020001	
NUM_ADC_MUX_OUTPUTS	3	Number of ADC MUX output pins.

1.6 SCML Commands

The following table describes the SCML commands implemented for SARADC module.

SCML Commands	Description	Arguments
set_calibration_status_offset	true = calibration process shall pass after configured time, false = calibration process shall fail	set_calibration_status <true/false>
set_calibration_status_capacitor	true = calibration process shall pass after configured time, false = calibration process shall fail	set_calibration_status <true/false>
set_self_test_data	sets the self test data, fractional data is only used for algorithm S	set_self_test_data<algo> <integer_Data> <fraction_data>

Chapter 2
AVPLL (PLL)

2.1 Introduction

Phase Locked Loop multiple instances are present to meet system clocking requirements. PLL contains PFD (Phase Frequency Divisor) and DFS (Digital Frequency Synthesis) that refer to a mechanism where a PLL lock to a high VCO frequency and multiple output clock frequencies are outputs through separate clock divisions from PLL. These output clock divisions are not necessarily integer divisions.
As a fractional-N synthesizer, PLL supports 30-bit numerator and denominator. Numerator is a signed number that can be used to adjust fractional portion of feedback divider dynamically. The PLL digital control unit provides a user interface and control of the PLL system. The PLL system consists of a single PLL analog block and a digital interface. PLL also have DFS blocks for fractional division with separated clock and status signals.

2.2 Supported Features

The AV_PLL model implements the following functionality:

- Programmable frequency modulation.
- Programmable output dividers.
- Capability to change runtime programmable fractional portion of feedback divider.
- Programmable clock bypass feature is available.
- Integer and fractional loop dividers
- Spread spectrum feature
- N number of DFS blocks supported
- DFS block has different output clocks and status signals

2.3 Features Not Supported

The PLL block will not implement the following functionality:

- Debug functionality is not implemented in the model.
- ANA_MFN in debug mode in status register is abstracted.
- All register setting where selection from register set and hardware selection is present are abstracted.

2.4 Assumptions

- Test register is not modelled as different test modes cannot be modelled. Debug mode is not modelled.
- The Input to the output-divider block default value of divider is taken as two.
- PLL1 instance selects safe clock when clksel bit value is 1.

2.5 SCML COMMANDS

Scml_command	Values passed	Description
set_fract_pll _lock	Value 0/1	Sets the value lock bit in FRACT_PLL if value passed is '1', lowers the loss of lock interrupt.
set_fract_pll_fail	Value 0/1	Sets the value LOL flag in FRACT_PLL if value passed is '1'.

Chapter 3
Body Cross-triggering Unit (BCTU)

3.1 Introduction

This chapter provides details on the Body Cross-triggering Unit (BCTU) model.

3.2 Supported Features

The BCTU block implements the following functionalities:

- 72 trigger inputs, configured with the trigger configuration (TRGCFG_0 - TRGCFG_71) registers.
- Trigger outputs to two ADCs.
- One priority selection logic block per ADC.
- Selection multiplexers to select one trigger at a time for each ADC.
- A conversion list of 32 ADC channels.
- A trigger grouping block to latch the hardware triggers. BCTU clears these triggers when the ADC sends an acknowledgment for conversion completion.

3.3 Features Not Supported

The BCTU block does not implement the following functionalities:

- MCR.FRZ functionality for Debug Freeze is not supported.
- Following interfaces are not supported:
  - ipg_hard_async_reset_b, ipt_scan_reset_b, are not modeled. p_reset_in signal is used for complete model reset.
- Following PSI related signals are not modeled.
  - ips_reacm_module_en
  - [15:0] ips_reacm_module_en
  - ips_reacm_rwb
  - [31:0] ips_reacm_wdata
  - [15:0] reacm_tx_wait

3.4 Assumptions

The following assumptions are taken while modeling this peripheral:

- You must always write 0 to reserved writable registers and fields. Read accesses of reserved registers and fields do not return meaningful data. For read from the reserved registers, model always returns 0 instead of non-meaningful data.
- Assuming FIFO word size to be 32 bits.
- Since there is no explicit module reset, p_reset_in is added.
- When MCR.Software_Reset is set, the model goes to reset. The model stays in reset state till MCR.Software_Reset is set by the software.
- There is no Trigger_clr output signal from BCTU for TRGMUX. But BCTU does not know source of trigger and the number of Trigger_clr is same as timer_trg. Hence, Trigger_clr output is assumed for even TRGMUX lines.
- If ips_module_en is de-asserted during operation of module is in running state, the module completes the current trigger request and any pending request and then goes to disable state. In disabled state, no trigger is serviced.
- To use a software trigger, you must first disable the corresponding hardware trigger (TRGCFG_-x[TRIGEN]). Software triggers operate regardless of individual trigger enable (TRGCFG_x[TRIGEN]).

Chapter 4
Central Self-Test Control Unit (CSTCU)

Supported Features
The model LSTCU supports the following features:

- All registers are modeled.
- Generic slave interface to access registers are modeled using tlm::tlm_target_socket<32>.
- Reset support is implemented.
- LBIST sequential/concurrent execution.
- MBIST sequential/concurrent execution.
- Programmable LBIST execution time.
- Configuring MBIST execution time using SCML command.
- Configuring LBIST execution time using SCML command.
- Status and end flags to signal BIST completion.
- LSTCU error UF/RF lines to CSTCU.
- BIST failure supported through SCML command.

4.1 Features Not Supported

The model LSTCU does not support the following features:

- Isolation, reset and indication signals not modeled.
- ISODUR, PH1_DUR, SRCENDUR registers related functionality not modeled.

4.2 SCML Commands

The table below describes the SCML parameters for the LSTCU model:

Command Name	Description
set_mbist_runtime_ns	This command is used to set the time required by a MBST to complete its execution. Argument 1 specifies the MBIST number whose execution time is to be set and argument 2 specifies the time in nano seconds.

set_bist_status	This command is used to set the status of BIST to pass/fail. The first argument specifies BIST is MBIST/LBIST, second argument specifies BIST number and the third argument specifies BIST will PASS/FAIL.
set_fbist_invalid_ptr	This command is used for injecting valid or invalid pointer error in FBIST mode.
Set_lbist_runtime_ns	This command is used to set the time required by a LBST to complete its execution. Argument 1 specifies the LBIST number whose execution time is to be set and argument 2 specifies the time in nano seconds.

Chapter 5
Clock Generation Module (MC_CGM)

5.1 Introduction

The clock generation module (MC_CGM) is used to set up the configurable clock domains used by various chip blocks as per the application needs. MC_CGM includes number of clock slices, which further includes clock multiplexers that allow software to select the desired clock sources for these domains.
This is managed by the MC_CGM to ensure that the changing of the clock selection from one source to another occurs in a glitch-less fashion. In addition, the MC_CGM includes the clock dividers that can be configured by software.

5.2 Supported Features

The MC_CGM block supports the following functionalities:

- Model reset supported. Implements multiple clock multiplexers for selecting various clock sources.
- Destructive reset and functional reset both are supported.
- Supports multiple input clock sources for selection for each clock multiplexer.
- Generates divided reference clock for on-chip resources using various clock dividers of clock multiplexers.
- Contains a set of registers to control clock dividers for divided clock generation.
- Safe Clock Switch feature is supported both through hardware and software.

5.3 Features Not Supported

The MC_CGM block does not support the following functionalities:

- Progressive clock switching for MC_CGM is not modeled. Progressive clock switching is used to prevent sudden voltage drops and overshoots due to frequency and load changes. In transaction level modeling it is assumed that these kinds of scenarios do not arise. Hence, this feature is not implemented. Corresponding registers only support read/write functionality.
- MC_CGM_DIV_UPD_STAT register handling when immediate divider update happens not modelled as all the dividers will get updated in zero time when Divider Control Register is updated.
- CSS.SWIP and CSC.SAFE_SW and CSC.CLK_SW not modelled as switching is done in zero time. Hence the value will remain 0. No progress can be shown.

- Glitch less clock transitions. In transaction level modeling it is assumed that these kinds of scenarios do not arise.

5.4 Assumption

- If the selected clock source is disabled, then the output clock derived from this source is also disabled.
- PCFS Register for clock source 0 ‘clk_src_0” is assumed as not required as it is not mentioned in the specification provided.
- For the Clock switch Trigger Cause (CSS.SWTRG), in case of failure of clock switch request, the current clock is checked first and if it is inactive, the CSS.SWTRG is set to value 3 and clk is switched to safe clk source (clk_src_0), and incase, the current clock is active, then the target clock is being checked.w

Chapter 6
Clock Monitoring Units (CMU)

6.1 Introduction

Clock Monitoring Units (CMU) are required on SoC to ensure sanity of clocks on which the SoC is running. The Frequency Check (FC) module checks if the frequency of an input monitored clock is within the frequency range specified by you. If the frequency is outside the specification limits, it could lead to performance, protocol, and timing failures. This module requires another clock which works as a base reference for the frequency check operation. This reference clock needs to be within the documented limits at all times, for correct operation of the module.
Supported Features
The virtual model in this release includes the following features:

- FLL and FHH event generation, when monitored clock frequency goes beyond low and high threshold respectively.
- Programmable duration of reference clock cycles.
- Time out functionality to generate FLL event, when the monitored clock is completely lost.
- Masking of FLL and FHH events.
- Support for on fly configuration.

6.2 Features Not Supported

The following features are abstracted out or not implemented:

- While monitoring, change in the duty cycle of the monitored clock is not detected.
- Loss of check through satellite checker is not implemented.

Chapter 7
CPE_GPR

Supported Features
The model CPE_GPR supports the following features:

- Configuration of the Cluster0 Cortex-R52 cores in the CPE subsystem.
- Status update whether Cortex-R52 Core1 and Core0 is in WFI low-power state.
- Status update whether Cortex-R52 Core1 and Core0 is in WFE low-power state.
- Cortex-R52 Core1 and Core0 warm core reset request.
- Core 0 and Core 1 software fault to FCCU.

7.1 Features Not Supported

The model CPE_GPR does not support the following features:

- Invalidation of L1 cache.
- Selection of TCMA local memory or flash memory as the boot location for the Cortex-R52 core_1.
- Selection of instruction set at reset.
- Configuration of Fence and Drain Enables and Isolation Enables for R52 cluster.
- System Counter Divider for Cortex-R52 processor Generic Timer.
- Enabling halt mode on Cortex-R52 Core1 and Core 0.
- Capturing the status of Source Synchronous Interface (SSI) Gasket Idle status (SSI_IDLE).
- Disabling PMC MBIST.
- Enabling and disabling Bus Parity Error checking.
- Configuring Source Synchronous Interface (SSI) Gasket.
- Updating MRAM ECC Data correction status.
- Updating status of events for Core0 and Core1.
- Primary and secondary memory error reporting interface memory identifier for Cortex-R52 Core0 and Core 1.
- Primary and secondary error reporting interface index value for Cortex-R52 Core0 and Core 1.
- Capturing the PMU event indicator outputs from 0-35 for both Core 0 and Core 1.

Chapter 8
Crossbar Integrity Checker (XBIC)

8.1 Introduction

The functionality of the XBIC is to detect errors in transactions from XBAR and it can also induce error in the transactions. Since XBAR is a stub model, error detection is implemented using SCML Commands. It also asserts error signal to FCCU.

8.2 Supported Features

The XBIC module implements the following features:

- Memory mapped bus interface for configuring the internal registers of XBIC.
- Error generation using SCML Commands on per port basis.
- Assertion of error to FCCU in case an error detected.
- Error injection is supported using SCML command.

8.3 Features Not Supported

The following features are not implemented in XBIC module:

- Bus phases and internal signals are abstracted.
- Functionality of DPME(0-7) and DPSE(0-7).
- XBIC model does not provide support for actual integrity checking of transactions, error can be generated or injected for a dedicated port using SCML command.

8.4 Assumptions

The following assumption is taken while modelling the XBIC module:

- MCR[SE5] is reserved, its reset value is based on a parameter and by default it will be read as 0.
- If Error injection (EIE) is enabled, the syndrome(SYN) value captured will be one updated in XBIC_EIR register instead of the one which is passed in SCML Command.

8.5 SCML Parameters

The following table describes the SCML parameters used by the XBIC module.

Name	Default Value	Description
PULSE_TIME	1	Duration in ns for which p_error_out pulse is asserted to FCCU.
MCR_RESET_VAL	0xFBFF0000	Since SE5 bitfield is reserved, its reset value is set to 0.

8.6 SCML Commands

The following table describes the SCML commands implemented for XBIC module.

SCML Commands	Description	Arguments
generate_error	Set error for configured port. First argument is the slave port, it's value can be (0 - 7). Second argument is the master port, it's value can be (0 - 7). Third argument is the syndrome value <SYD>. Enter SYD in hex Fourth argument is the address for which integrity error is generated <adddress>. Enter address in hex. Example – detect_error "1" "2" "0xAA" "0xABCDACAD"	<slave_port><master_-port><syndrome><address> Set error for configured port. First argument is the slave port, it's value can be (0 - 7). Second argument is the master port, it's value can be (0 - 7). Third argument is the syndrome value <SYD>. Enter SYD in hex. Fourth argument is the address for which integrity error is generated <adddress>. Enter address in hex. Example – detect_error "1" "2" "0xAA" "0xABCDACAD"
inject_error	Set error for configured port(master and slave) in EIR reg. Running this command is same as a passing a transaction to XBIC in real hardware. First argument is the address for which error is injected <adddress>. Enter address in hex.	<address>

Chapter 9
Cyclic Redundancy Check (CRC)

9.1 Introduction

This chapter provides the design specifications for the Supplemental Register Module for NXP platform, as a SystemC TLM2/SCML2 compliant model. This is modeled at LT abstraction level with timing annotation. The intended use case of this model is software development. In this design we shall try to separate behavior, communication and timing as far as possible. The functional description of Cyclic Redundancy Check (CRC) along with the information about the internal registers and interface ports are discussed in detail.

9.2 Supported Features

The CRC module implements the following features:

- All register modeled.
- Register memory modeled using scml2::memory<unsigned>.
- Generic slave interface to access registers modeled using scml2::ft_target_socket<32>.
- Hardware CRC generator circuit sing a 16/32 bit programmable shift register.
- Soft reset supported.
- Programmable Initial seed value and polynomial.
- Option for inversion of final CRC result.
- Option to transpose input data or output data (CRC result) bitwise or bytewise. This option is required for certain CRC standards. A bytewise transpose operation is not possible when accessing the CRC data register via 8-bit access. In this case your software must perform the bytewise transpose function.

Chapter 10
De-serial Serial Peripheral Interface (DSPI)

10.1 Introduction

The module is a DSPI device, which supports full-duplex, synchronous, serial communication between MCUs and peripheral devices. The DSPI module generally sits between the System bus (for communication with Host/DMA and so on) and the SPI bus (for serial transfers). The DSPI module can operate in the following three configurations.

- SPI Configuration: The SPI interface allows the DSPI to send and receive serial data. The SPI block has internal FIFOs supporting external queues operation. The host CPU or DMA controller read the received data from receive FIFO and write transmit data to the transmit FIFO.
- DSI Configuration: In the DSI configuration the DSPI serializes up to 64 parallel input signals or register bits. The DSPI also deserializes the received data to parallel output signals or to a memory mapped register.
- CSI Configuration: These are the configurations available in this mode.
  - The normal CSI mode is a combination of the SPI and DSI configurations.
  - In the TSB mode, the SPI data has higher priority than DSI data.
  - In the ITSB mode, the frames from SPI and DSI are interleaved without priority.

10.2 Supported Features

This DSPI model will implement the following functionality:

- All registers modeled.
- Memory mapped slave interface to configure the internal registers of DSPI using tlm::tlm_-target_socket <32>.
- Serial data RX/TX interfaces implemented as generic serial socket. Customized serial payload used for CSIH Tx/Rx frame.
- Support for full-duplex, three-wire synchronous transfers that is, both transmit and receive paths to be active simultaneously.
- Support for master and slave modes operation.
- Four entry TX FIFO and RX FIFO buffers are supported.
- Disabling feature for TX and RX FIFOs are supported.
- Three configurations are supported:
  - SPI

  - DSI
  - CSI
- Programmable transfer attributes on a per-frame basis
  - 8 transfers attribute registers.
  - Programmable serial frame size of 4 to 32 bits.
  - Programmable master bit rates.
- Six peripheral chip selection is done using command frame in PUSHR register.
- DMA support for managing TX and RX FIFOs:
  - TX FIFO fill (TFFF).
  - RX FIFO Drain (RFDF).
- Support for interrupt generation. Nine different sources:
  - End of Queue (EOQ).
  - TX FIFO fill (TFFF).
  - Transfer Complete (TCF).
  - TX FIFO underflow (TFUF).
  - RX FIFO Drain (RFDF).
  - RX FIFO overflow (RFOF).
  - SPI Parity Error (SPEF).
  - DSI Parity Error (DPEF).
  - DSI Deserialized data Match (DDIF).
- Programmable transfer baud rate.
- Support for power saving mode.
  - Stop mode.

This DSPI model supports serialization and deserialization if enabled for the module. DSPI model implements the following functionality for DSI.

- Selection of serialized data through following sources, except for DSPI_E. For DSPI_E, data can be only selected from DSPI Memory mapped register (DSPI_ASDR0):
  - DSPI memory mapped register.
  - Parallel input signals.
- Deserialized data provided as:
  - Parallel output signals.
  - DSPI memory mapped register bits.
- Support for continuous and non-continuous transfer.

10.3 Features Not Supported

The DSPI block does not support the following functionalities:

- TSB (Timed Serial Bus) is not modeled as this functionality is not necessary now.
- Clock Phase and data phase management.
- Various delays like PCS to SCK delay, after SCK delay.

10.4 Assumptions

- Exact clock phase/polarity at which the transmission happens is not of relevance, at the LT level of abstraction.
- Set-up and hold time delays and the delays between two frame transfers, will not be used in delay calculations.
- A DSPI master could well be communicating to “nothing” (that is, no DSPI slave in RUNNING state) and not know it. But, in the LT style of modeling, we will at-least print appropriate messages and set “tlm_status_response” as TLM_INCOMPLETE, to help you easily identify such issues and fix them.
- When the data is received by the DSPI slave, it is important that slave is in “Running State”, if it’s not the case, we will flag an error.
- We’ll not support multi-master/multi-slave set-up on SPI bus.

Chapter 11
Direct Memory Access Multiplexer (DMAMUX)

11.1 Introduction

The Direct Memory Access Multiplexer (DMAMUX) routes DMA sources, called slots, to any of the 16 DMA channels. On every memory map configuration change for any channel, this module signals to the DMA Controller to reset the internal state machine for that channel and it can accept a new request based on the new configuration.

11.2 Supported Features

The DMAMUX module implements the following features:

- Memory mapped bus interface for configuring the internal registers of DMA_MUX.
- Memory mapped 32-bit configuration registers for each of the 16 DMA channels.
- Channel service requests from 28 hardware peripherals for starting a DMA transfer.
- 4 trigger inputs from the PIT to be used in Periodic Trigger Mode of channels 0-4.
- Up to 92 peripheral slots routed to 16 channels.

Chapter 12 DMA Cyclic Redundancy Check(DMA_CRC)

12.1 Introduction

DMA CRC is a plugin model that latches on TLM2 DMA transactions originating from the eDMA3 module and calculates CRC checksum on the read transaction's payload.

12.2 Supported Features

The DMA_CRC module implements the following features:

- TLM2 generic payload Interface for accessing CRC's control and status registers. The CRC control register is set prior to selecting the functionalities of the CRC, such as enabling/disabling it, selecting the polynomial, preloading of the CRC and so on.
- Supports up to 8 channels. The number of CRC channels in the model is configurable using SCML2 parameter.
- Supports up to 6 different polynomials (8 bit, 16 bit, and 32 bit). Each channel has its own polynomial or share the same polynomial. Below are the polynomials being supported:
  - Ethernet (IEEE 802,3) CCITT32 CRC-32 0x04C11DB7.
  - Ethernet (IEEE 802,3) CCITT32 CRC-32 0x04C11DB7.
  - E2E Profile 4 CRC-32 P4 0xF4ACFB13.
  - x.25. CCITT16 CRC-16 0x1021.
  - H2F, Autosar 4.0 CRC-8 0x2F.
  - VDA CAN, SAE-J1850 CRC-8 0x1D.
- CRC functionality enabled/disabled per channel.
- Global CRC enabled to safeguard against accidental enablement.
- CRC register preloaded at the start of the DMA transfers with an Intial Value (ICRC register) or continued with the accumulated CRC value from previous read transactions. Software preloads the ICRC register with a pre-calculated checksum value, and checks for a zero checksum value at the end of the DMA transfers.
- The intermediate and final result of the CRC is accessible through the CSR (CRC Status Register). Each channel has its own CSR.
- Supports Read only and Write only DMA transfers. In Read only mode, the DMA controller suppresses the Write Transfers. In Write only mode, the DMA controller suppresses the Read transfers.

- Supports Register Transfer List Write and Register Transfer List Read mode. In Register Transfer Write mode, the 64 bit read data contains a pair of <32bit address, 32 bit write data>.
- All capabilities of current eDMA are still supported, such TCDs, channel Linking, scater-gather and so on.

12.3 Features Not Supported

The DMA_CRC block does not support the following functionalities:

- Due to a limitation of the scml2::router constructor of SCML2 library, the address range of DMA transactions is assumed to be limited to 32-bits. 32-bit address is sufficient for the current needs, and can be extended in the model, but the scml2::router API in scml2 does not allow mapping a complete 64-bit address range.

12.4 Assumptions

- Address bus of the DMA transaction is assumed to be 32-bit.

Chapter 13
DSP_GPR

13.1 Supported Features

The model DSP_GPR supports the following features:

- Lock enable functionality for blocking the transaction from S3 interface to reach to PERIPHERAL XBAR.
- Processor can enter halt and stop state.
- Wait, Stop and Halt status of DSP-V CPU updated.

13.2 Features Not Supported

The model DSP_GPR does not support the following features:

- AXBS and processor reset information is missing.

Chapter 14
DSP-V Subsystem (DSP-V)

14.1 Introduction

This chapter describes the SubSystem p_ip_zdv_syn which includes the ZV2790 Core (DSP-V, EZH-V and RISC-V), an interrupt controller (INTC), SWT and STM timers, three RAM memory interfaces, as well as four Message Units.

14.2 Supported Features

The ZENV block implements the following functionalities:

- 32- BIT RISC v compliant CPU.
- Supports following sets of instructions defined by RISC-V International.
- RV 3 2I 32 bit integer instruction set.
- C standard extension compressed instruction set.
- M standard extension- multiply/ divide instruction set.
- ZICSR standard extension- control/status register modification instruction set.
- Zbb standard extension- basic bit manipulation instructions.
- ZCB standard extension-basic code size saving instructions.
- ZCMP standard extension I basic code size saving instructions.
- Floating point instructions.
- Support custom extensions.
- Branch extension (XZ branch).
- Message send/receive extension (XZMSG).
- Bit field extension (XZBF).
- DSP-V extension (XZDSP).
- EZH- vh extension (XZEZBY).
- Little Endian.
- Precise exception handling.
- Imprecise exception support.
- 32 entry 32 bit general purpose register file.
- 32 bit instruction site and 32 bit data site system interface.

14.3 Features Not Supported

The ZENV model has following limitations:

- Trace and debug features are not supported.
- slv_reset_b and nex_reset_b interface signals are not supported.
- Master ID configuration signals (p_masterid_t, nex_masterid) are not supported.
- Following sets of instructions defined by RISC-V International are not supported:
- Zcmt Standard Extension - Basic code size saving instructions.
- Following power management signals are not supported:
- p_halt, p_halted, p_stop, p_stopped, p_wakeup.
- Following processor state signals are not supported:
- p_reg_priv_t, p_mod_priv_t
- p_mtbucsr_t, p_mtl1csr0_t, p_mtl1csr1_t, p_mte2ectl0_t, p_mte2eecsr0_t, p_mtdmemctl0_t, p_mtimem-ctl0_t.
- p_thread_enabled_t
- p_mode
- p_brstat_t
- p_pstat_pipe0, p_pstat_pipe1
- p_syscmp_t
- ECC error indicator signals are not supported.
- Performance monitor signals are not supported.
- Memory synchronization signals are not supported.
- Following EZHV and DSPV signals are currently stubbed:
- ezhv_gpio_out
- ezhv_gpio_dir
- ezhv_int_ind
- ezhv_int_chan
- ezhv_gpio_in
- ezhv_blc_in

Chapter 15
ENET_PHY_WRAPPER (SERDES)

Supported Features
The model ETH_PHY_WRAPPER supports the following features:

- 1G SGMII (1.25 Gbit/s) with Clause37 Auto Negotiation.
- SGMII 10/100 Mbps with Clause 37 Auto Negotiation.
- 2.5G SGMII (overclocked SGMII - 3.125 Gbit/s).
- 1000Base-KX with Clause 73 AN.
- 2.5GBase-KX with Clause 73 AN (3.125Gbps).
- EEE (energy-efficient Ethernet).

15.1 Features Not Supported

The model ETH_PHY_WRAPPER does not support the following features:

- SFD (start frame delimiter) detect as this is a physical level feature.

Chapter 16
Enhanced Direct Memory Access (eDMA)

16.1 Introduction

The Enhanced Direct Memory Access (eDMA) controller is a third-generation module capable of performing complex data transfers with minimal intervention from a host processor. The hardware micro-architecture includes:

- A DMA engine that performs:
  - Source and destination address calculations.
  - Data movement operations.
- Local memory containing transfer control descriptors for each of the 32 channels.

16.2 Supported Features

The eDMA3 module implements the following features:

- All data movement through dual-address transfers - read from source, write to destination.
  - Programmable source and destination addresses and transfer size.
  - Support for enhanced addressing modes.
- 32-channel implementation that performs complex data transfers with minimal intervention from a host processor.
  - Internal data buffer, used as temporary storage to support 16-, 32-, and 64-byte transfers.
  - Connections to the crossbar switch for bus initiating the data movement.
- Transfer Control Descriptor (TCD) organized to support two-deep, nested transfer.
- Operations:
  - 32-byte TCD stored in local memory for each channel.
  - An inner data transfer loop defined by a minor byte transfer count.
  - An outer data transfer loop defined by a major iteration count.
- Channel activation via one of three methods:
  - Explicit software initiation.
  - Initiation via a channel-to-channel linking mechanism for continuous transfers.
  - Peripheral-paced hardware requests, one per channel.
- Fixed-priority and round-robin channel arbitration.
- Channel completion reported via programmable interrupt requests.

  - One interrupt per channel, which can be asserted at completion of major iteration count.
    - Error terminations per channel and logically summed together to form one error interrupt to the interrupt controller.
- Programmable support for scatter/gather DMA processing.
- Support for complex data structures.

16.3 Features Not Supported

The eDMA3 block does not support the following functionalities:

- DMA CRC signals ae abstracted out, as a Virtual Prototype does not have CRC errors.
- Lockstep is abstracted out, as it is not required in a loosely-timed Virtual Prototype.
- Functionality of ATTR bitfield of CHa_SBR register is abstracted out.

16.4 Assumptions

- Only one error interrupt output signal has been modeled for all the channels/groups.
- Channel and group priority are changed after setting MP_CSR[HALT] bit.
- It is assumed that the ipd_req signal from any hardware requesting the channel service is cleared by hardware or software and not by the eDMA.
- If ipd_req is still asserted for the channel which has completed the minor loop, then channel is added in arbitration process and is served again.
- When transfer has been canceled due to dma_err_cancel_xfer/MP_CSR.ECX, then:
  - Error is set.
  - TCD is updated with source, destination address and citer values.
  - Execution stops.
  - The minor loop post processing does not get executed in this case.
- If bus error occurs during one read/write sequence, then further reading/writing is stopped immediately whether the error happened in read or write.
- If any bus error occurs, after setting the error and updating TCD, execution stops. The minor/major loop post processing will not get executed in this case.
- If any configuration error occurs, after setting the error, execution would stop. The minor/major loop post processing will not get executed in this case. No CITER count will be decreased. However, dma_ipd_done and dma_done_lw signals gets updated.
- eDMA reads back current value of TCDa_SADDR, TCDa_DADDR and TCDa_NBYTES if channel is active.
- TCD.done corresponds to CHa_CSR.done bit.
- UCE error can be generated using SCML command.

16.5 SCML Objects

The following table provides detailed information on SCML parameters supported by eDMA3.

Name	Value	Description
STATE	EDMA_RESET_-STATE EDMA_HALT_-STATE EDMA_IDLE_-STATE EDMA_AC-TIVE_STATE	This status object displays current state of eDMA3 module.

16.6 SCML Commands

The table below provides detailed information on SCML commands supported by eDMA3.

SCML Commands	Description	Arguments
set_uce_bit	SCML command to generate UCE error. Example: set_uce_bit <value> <channel_num> If value passed is 1, then UCE bit is set to 1 else UCE bit is set to 0 and channel_num argument is optional.	<value> <channel_num>

Chapter 17
Enhanced Direct Memory Access (eDMA5)

17.1 Introduction

The *Enhanced Direct Memory Access (eDMA)* controller is a third-generation module capable of performing complex data transfers with minimal intervention from a host processor. The hardware micro-architecture includes:

- A DMA engine that performs:
  - Source and destination address calculations.
  - Data movement operations.
- Local memory containing transfer control descriptors for each of the 32 channels.

17.2 Supported Features

The eDMA4 module implements the following features:

- 32 channels for transferring the data from memory to memory, memory to peripheral, peripheral to memory, peripheral to peripheral. Total number of channels can be configured.
- Memory mapped bus interface for configuring the internal registers and the TCD block of eDMA.
- Initiator socket for data transfer operation.
- Parameterized support for 32- and 64-bit AMBA-AHB data path widths.
- All data movement through dual-address transfers: read from source, write to destination.
- Programmable source and destination address and transfer size.
- Support for enhanced addressing modes.
- Channel activation through one of three methods:
  - Explicit software initiation.
  - Initiation through a channel-to-channel linking mechanism for continuous transfers.
  - Peripheral-paced hardware requests, one per channel.
- Transfer control descriptor (TCD) organized to support two-deep, nested transfer operations.
- 32-byte TCD stored in local memory for each channel.
- An inner data transfer loop defined by a minor byte transfer count.
- An outer data transfer loop defined by a major iteration count.

- One interrupt per channel, which can be asserted at completion of major iteration count.
- Programmable error terminations per channel and logically summed together to form one error interrupt to the interrupt controller.
- Programmable support for scatter/gather DMA processing.
- Support for complex data structures.
- Fixed priority and Round Robin channel arbitration.
- Channel Pre-emption when a higher priory channel gets a request.
- Major loop and minor loop channel linking.
- Scatter and gather functionality.
- Support to cancel transfers.
- Bandwidth throttling.
- Byte swapping.
- Sign extension.
- Hardware service requests for 128 inputs.
- Channel multiplexing based on hardware service requests.

17.3 Features Not Supported

The following features are abstracted out or not implemented in eDMA4 module:

- DMA CRC signals are abstracted out, as a Virtual Prototype does not have CRC errors.
- Lockstep is abstracted out, as it is not required in a loosely-timed Virtual Prototype.
- Functionality of ATTR bitfield of CHa_SBR register is abstracted out.
- Functionality of CHn_MATTR register has been abstracted out.

17.4 Assumptions

- It is assumed that the ipd_req signal from any hardware requesting the channel service is cleared by hardware or software and not by the eDMA.
- If ipd_req is still asserted for the channel which has completed the minor loop, then channel will be added in arbitration process and will be served again.
- When transfer has been cancelled due to dma_err_cancel_xfer/MP_CSR.ECX, then error is set, TCD updated with src and destination address and citer values and execution stops. The minor loop post processing does not get executed in this case.
- If bus error occurs during one read/write sequence, then further reading/writing is stopped immediately whether the error happened in read or write.
- If any bus error occurs, after setting the error and updating TCD, execution would stop. The minor/major loop post processing will not get executed in this case.

- If any configuration error occurs, after setting the error, execution would stop. The minor/major loop post processing will not get executed in this case. No CITER count will be decreased. However, dma_ipd_done and dma_done_lw signals will get updated.
- eDMA reads back current value of TCDa_SADDR,TCDa_DADDR and TCDa_NBYTES if channel is active.
- UCE error can be generated using SCML command.

17.5 SCML Objects

The following table describes the SCML parameters used by the eDMA4 module.

Name	Value	Description
STATE	EDMA_RESET_STATE EDMA_HALT_STATE EDMA_IDLE_STATE EDMA_ACTIVE_- STATE	This status object displays current state of eDMA4 module.

17.6 SCML Commands

The following table describes the SCML commands implemented for eDMA4 module.

SCML Commands	Description	Arguments
set_uce_bit	SCML command to generate UCE error. **Example:** *set_uce_bit* <*value*> <*channel_num*> If value passed is 1, then UCE bit is set to 1 else UCE bit is set to 0 and channel_num argument is optional.	<*value*> <*channel_num*>

Chapter 18
Enhanced Direct Memory Access Cyclic Redundancy Check (eDMA3_CRC)

18.1 Introduction

DMA CRC is a plugin model that latches on TLM2 DMA transactions originating from the eDMA3 module and calculates CRC checksum on the read transaction's payload.

18.2 Supported Features

The DMA CRC block supports the following functionalities:

- TLM2 generic payload interface for accessing CRC's control and status registers. The CRC control register has to be set prior to selecting the functionalities of the CRC, such as enabling/disabling it, selecting the polynomial, preloading of the CRC and so on.
- Supports up to eight channels. The number of CRC channels in the model is configurable using SCML2 parameter.
- Supports up to six different polynomials (8 bit, 16 bit, and 32 bit). Each channel can have its own polynomial or share the same polynomial. Below are the polynomials being supported:
  - Ethernet (IEEE 802,3) CCITT32 CRC-32 0x04C11DB7.
  - Ethernet (IEEE 802,3) CCITT32 CRC-32 0x04C11DB7.
  - E2E Profile 4 CRC-32 P4 0xF4ACFB13.
  - x.25. CCITT16 CRC-16 0x1021.
  - H2F, Autosar 4.0 CRC-8 0x2F.
  - VDA CAN, SAE-J1850 CRC-8 0x1D.
- CRC functionality can be enable/disable per channel.
- Global CRC enable to safeguard against accidental enablement.
- CRC register can be preloaded at the start of the DMA transfers with an intial value (ICRC register) or continued with the accumulated CRC value from previous read transactions. Software can preload the ICRC register with a pre-calculated checksum value and check for a zero checksum value at the end of the DMA transfers.
- The intermediate and result of the CRC is accessible through the CSR (CRC Status Register). Each channel has its own CSR.
- Supports Read only and Write only DMA transfers. In Read only mode, the DMA controller suppresses the Write Transfers. In Write only mode, the DMA controller suppresses the Read transfers,

- Supports Register Transfer List Write and Register Transfer List Read mode. In Register Transfer Write mode, the 64 bit read data contains a pair of <32bit address, 32 bit write data>.
- All capabilities of current eDMA are still supported, such TCDs, Channel Linking, Scatter-Gather, and so on.

18.3 Features Not Supported

The DMA CRC block does not support the following functionalities:

- Due to a limitation of the scml2::router constructor of SCML2 library, the address range of DMA transactions is assumed to be limited to 32-bits. 32-bit address is sufficient for the current needs, and can be extended in the model, but the scml2::router API in scml2 does not allow mapping a complete 64-bit address range.

18.4 Assumptions

- Address bus of the DMA transaction is assumed to be 32-bit.

Chapter 19
Error Injection Module (EIM)

19.1 Introduction

The Error Injection Module (EIM) provides support for inducing single-bit and multi-bit inversions on read data when accessing peripheral RAMs. Injecting faults on memory accesses can be used to exercise the SEC-DED ECC function of the related system.

19.2 Supported Features

The EIM module implements the following features:

- Provides support for inducing single-bit and multi-bit inversions on read data when accessing peripheral RAMs.
- Supports 23 error injection channels.
- Protection against accidental enable and reconfiguration error injection function through two-stage enable mechanism.

19.3 Features Not Supported

The following features are abstracted out or not implemented in EIM module:

- Being as a LT model, EIM raises ECC error event to ERR model instead of data inversion on memory read
- To maintain future scalability of model, access on undefined (reserved) memory addresses does not generate an IPS error.

19.4 Assumptions

The following assumption is taken while modelling the EIM module:

- ECC event raised to ERR model on configuring error injection for a channel.
- You need to update eim_probe.py script [in shared directory] to customize EIM-ERM mapping.
- EIM individual channel configuration has been done as per specification RM638611-S32Z27 ref Manual(1.1); that is, 31 Channel; 0 to 30 with different Data Mask and Channel Mask bit widths.

19.5 SCML Parameters

The following table describes the SCML parameters used by the EIM module.

Name	Default Value	Description
NUM_EIM_CHANNELS	2	Number of EIM channels.
CHKBITMASKWIDTHCH0 to 31	32	Number of CHKBITMASK for each channel.
DATAMASKWIDTHCH0 to 31	32	Number of DATAMASK for each channel.

Chapter 20
Error Reporting Module (ERM)

20.1 Supported Features

The model ERM supports the following features:

- Support for ECC event capturing for memory sources with individual reporting field and fault configuration per memory channel.
- Optional fault notification on captured ECC events.
- Registers only programmable in supervisor mode and using only 32 bit (word) accesses.
- Recording of the count value of the number of correctable error events.
- Capturing of address and syndrome information on single-bit correction and non-correctable ECC events.

20.2 Assumptions

ERM interrupt/fault notifications are connected to FCCU rather than GIC. The model specification describes capability for interrupt notification although the fault connectivity sheet has entries for connection from ERM to FCCU. Hence following connectivity has been implemented in VDK as per Fault Connection List.

ERM Instruction	Channel Number	Memory	FCCU Fault
ERM PER	0	FlexRay_0 LRAM	NCF[52]
	1	FlexRay_0 DRAM	NCF[52]
	2	FlexCAN_0	NCF[52]
	3	FlexCAN_1	NCF[52]
	4	FlexCAN_2	NCF[52]
	5	FlexCAN_3	NCF[52]
	6	Quad SPI RX	NCF[70]
	7	Quad SPI TX	NCF[70]
	8	SRAMC	NCF[14]
ERM eDMA0	0	eDMA0 TCD	NCF[43]
ERM eDMA1	0	eDMA1 TCD	NCF[45]

20.3 SCML Commands

The table below describes the SCML parameters for the ERM model:

Command Name	Description
triggerECCErrEv	Command to trigger an ECC error event for Nth Memory Channel.

Chapter 21
Extended Resource Domain Controller (XRDC)

21.1 Introduction

The Extended Resource Domain Controller (XRDC) provides an integrated, scalable architectural framework for access control, system memory protection and peripheral isolation. To support enforcement of robust operational environments, it allows software to assign chip resources including processor cores, non-core bus masters, memory regions and target peripherals, to processing domains.

21.2 Supported Features

The XRDC block implements the following functionalities:

- XRDC module includes:
  - The Manager submodule (MGR)
  - The Master Domain Assignment Controller (MDAC)
  - The Memory Region Controller (MRC)
  - The Peripheral Access Controller (PAC)
- Assignment of chip resources to processing domains:
  - Processor cores, non-core bus initiators, target memories and target peripherals.
  - Each processing domain is assigned a unique domain identifier (domain ID, DID).
  - Domain ID is a new attribute associated with every system bus transaction.
  - Used in conjunction with user/privileged, secure/non-secure attributes.
- Access rights to targets defined in region descriptor registers for memories and access control registers for peripherals.
- Supports sharing of memory and peripherals with optional inclusion of hardware semaphores to dynamically determine access rights.
- Built upon a 4-level hierarchical access control model:
  - SecurePriv
  - SecureUser
  - NonsecurePriv
  - NonsecureUser
- Memory region descriptor management.
- Domain error capture management.

21.3 Features Not Supported

The XRDC block does not implement the following functionalities:

- XBAR is abstracted out.
- Virtualization aware (VAW) and LPID are not modeled.
- Cache coherency interconnect (CCI)

Chapter 22
External Serial Peripheral Interface (XSPI)

22.1 Supported Features

The model XSPI supports the following features:

- Flexible sequence engine.
- One serial flash device can be connected and accessed in parallel for data read operations, forming one (virtual) flash memory with doubled readout bandwidth.
- AHB initiator to read RX buffer data through AMBA AHB (64-bit width interface) or IP registers space (32-bit access) and fill TX buffer through IPS register space (32-bit access).
  - AHB initiator can be a DMA with a configurable inner loop size.
- Multiple interrupt conditions.
- Current model supports only little endian.
- All AHB accesses to flash devices are directly memory mapped to the chip system memory.
- Programmable sequence engine to cater to future command or protocol changes and ability to support all existing vendor commands and operations. The software needs to select the corresponding sequence according to the connected flash device.
  - Supports all types of addressing.
- Support for Macronix Octal Data integrity features such as ECC and R/W parity (CRC1): This feature is abstracted out, but SCML commands are provided to assert bits in register FR, RSER.
  - AHB buffer implementation.
- Secure flash protection.
- Inline Prince Encryption and Decryption START/END status register update functionality implemented.

22.2 Features Not Supported

The model XSPI does not support the following features:

- Data learning feature:
  - The reason is data learning is not implemented in the initial version.
- Single, dual and quad modes of operation supported for quad flashes.
- QuadSPI serial interface is abstracted out with higher level interface (TLM2.0).
- Octal and single IO modes of operation supported for octal flashes.

- Double Data Rate (DDR) or Double Transfer Rate (DTR) mode wherein the data is generated on every edge of the serial flash clock.
- Flash timing registers (FLSHCR) is implemented but functionality is abstracted out.
- Support for parallel writes through register-mapped interface in single I/O mode.
  - The reason is because of no timing restriction in LT modelling.
- Flash data strobe signal to support data sampling in DDR and Single Data Rate (SDR) mode.
  - The reason is the SPI interfacing is abstracted to a higher-level interface.
- Multi-initiator accesses:
  - The reason is that only one initiator has access to QuadSPI at a time.
- Support for interfacing with On-The-Fly_Decription-Engine (OTFAD).
- AHB write access is not implemented in initial level model.
- Support for hyperflash.
- Support for hyperRAM.
- DMA to read data from Rx buffer:
- IPED functionality not implemented.
- AHB burst read/write feature is not implemented.
- Serial domain Soft Reset functionality.

22.3 Assumptions and Dependencies

- If LUT is programmed with invalid instructions, then invalid instructions are sent to external flash.
- In case of JMP_TO_SEQ, if operand points to previously executed instruction, then it goes in infinite loop.
- Data from tx buffer is popped out in case write instruction is sent to external flash.
- Watchdog timeout can be asserted by sending SET_WDOG_TIMEOUT SCML command to QuadSPI.
- If number of byte access through AHB interface are more than AHB buffer size, then bytes in next AHB buffer is overwritten.
- Watchdog is reset automatically after asserting error flag and user needs to resend it to raise the error flag.

22.4 SCML Commands

The table below describes the SCML parameters for the XSPI model:

Command Name	Description
set_ecc_int_A	This command is used to set the ECC interrupt for the flash memory A.

Chapter 23
Fast Crystal Oscillator (FXOSC)

23.1 Introduction

The FXOSC (Fast Crystal Oscillator) Digital interface is used to control the oscillator and provide the register interface for the programmable features. The FXOSC power down can be done by the software.

23.2 Supported Features

The FXOSC block supports the following functionalities:

- Generates 8/16/40 MHz clock output in Functional mode (or Normal mode).
- Contains Status register which is automatically set to one when crystal start-up time is completed.

23.3 Features Not Supported

The FXOSC block does not support the following functionalities:

- Hysteresis Control as this feature is abstracted out at TLM level.
- Automatic level controller is not modelled.
- Oscillator is having output signal RAW clock from analog for observational purpose as it provides the digital clock signal only and analog signal is abstracted out at the TLM level.

23.4 Assumptions

- Internal Oscillator Clock (Crystal) frequency and EXTAL and XTAL clock frequency is set to 8 MHz.
- For FXOSC CTRL register, OSC_BYP bit is cleared by writing 0 to it which will change the mode to Normal Mode and update clock when OSCON bit is 1.

Chapter 24
Flex Timer Module (FTM)

24.1 Introduction

The FlexTimer module (FTM) is a two-to-eight channel timer that supports input capture, output compare, and the generation of PWM signals to control electric motor and power management applications. The FTM time reference is a 16-bit counter that can be used as an unsigned or signed counter.

24.2 Supported Features

The FTM supports the following functionalities:

- FTM source clock is selectable.
  - The source clock is either system clock or an external clock.
  - Fixed frequency clock is an additional clock input to allow the selection of an on-chip clock source other than the FTM input clock.
  - Selecting external clock connects FTM clock to a chip level input pin therefore allowing to synchronize the FTM counter with an off-.chip clock source.
- Pre-scaler divide-by 1, 2, 4, 8, 16, 32, 64, or 128.
- 16-bit counter.
  - It can be a free-running counter or a counter with initial and final value.
  - The counting can be up or up-down.
- Each channel can be configured for input capture, output compare, or edge-aligned PWM mode
- In Input Capture mode:
  - The capture can occur on rising edges, falling edges or both edges.
  - An input filter can be selected for some channels.
- In Output Compare mode the output signal can be set, cleared, or toggled on match.
- All channels can be configured for centre-aligned PWM mode.
- Each pair of channels can be combined to generate a PWM signal with independent control of both edges of PWM signal.
- The FTM channels can operate as pairs with equal outputs, pairs with complementary outputs, or independent channels with independent outputs.
- The deadtime insertion is available for each complementary pair.
- Generation of match triggers.

- Initialization trigger.
- Software control of PWM outputs.
- The polarity of each channel is configurable.
- The generation of an interrupt per channel.
- The generation of an interrupt when the counter overflows.
- The generation of an interrupt when a register reload point occurs.
- Synchronized loading of write buffered FTM registers.
- Half cycle and full cycle register reload capacity.
- Write protection for critical registers.
- Backwards compatible with TPM.
- Testing of input captures for a stuck at zero and one condition.
- Dual edge capture for pulse and period width measurement.
- Quadrature decoder with input filters, relative position counting, and interrupt on position count or capture of position count on external event.
- Up to four fault inputs for global fault control.
- The FTM channels can be selected to generate a trigger pulse on channel output instead of a PWM.

24.3 Features Not Supported

The FTM block does not support the following functionalities:

- BDM mode.

24.4 Assumptions

- Value of counter (via FTM_CNT) would be read only on system clock edges and might be inaccurate in between.
- Value of PWM synchronized registers and their buffers cannot be examined even via VPX, before the synchronization happens.
- In Input Capture mode, if ICRST bit is set, the pre-scaler clock is not reset.
- All the triggers and pulses are clocked with System clock.

Chapter 25
Flexible Data Rate CAN (FlexCAN)

25.1 Introduction

The FlexCAN module is a communication controller implementing the CAN protocol compliant to CAN 2.0B protocol specification.

25.2 Supported Features

The FlexCAN module implements the following features:

- AHB Slave interface will be implemented as TLM2.0 target socket.
- CAN RX/TX interfaces will be implemented as generic serial socket respectively. Generic Serial Payload will be used for CAN Frame specific fields.
  - Memory mapped address space.
  - All of the address space will be accessible through the slave interface. The address space includes 4 internal memories. CAN Register, CANFD Register, Message Buffer and Individual Masks.
  - All access restriction specified in the data sheet.
  - Supervisor and User mode access restrictions.
- Following modes of operation will be supported.
  - Normal Mode.
  - Loop Back Mode.
  - Freeze Mode.
  - Disable Mode.
  - Stop Mode.
- Backward compatibility features.
- Message Buffers.
  - Number of message buffer to support will be configurable through scml property.
    - Extra IFLAG and IMASK register will be added if it is more than 32. This space will be reserved otherwise.
    - RXIMR register will be added or removed and remaining space will be made reserved.
    - Configured number of interrupts will be functional.
  - RX FIFO functionality.
- TX Functionality.
  - Message buffer Arbitration.

  - Abort.
  - CAN Frame transmission.
  - Acknowledge error.

- RX Functionality.
  - CAN Frame reception .
  - message buffer Matching .
  - Locking/unlocking mechanism.
- Clock source selection.
- Free Running timer.
- Message buffer interrupts.
- Block status interrupts.
- Hard and soft resets.
- CAN Protocol
  - Standard frames.
  - Extended frames.
  - Data frames.
  - Remote frames.
  - Error frames.

25.3 Features Not Supported

The following features are abstracted out or not implemented in FlexCAN module:

- Message buffer deactivation for TX and RX
- BUSY bit in message buffer CODE for TX and RX
- The following register bit fields have no functionality:
  - CTRL.RJW.
  - CTRL.PSEG1.
  - CTRL.PSEG2.
  - CTRL.SMP.
  - CTRL.LOM.
  - CTRL.ROM.
  - CTRL.PROPSEG.
  - ECR.
  - ESR.ACK_ERR.
  - ESR.CRC_ERR.
  - ESR.FRM_ERR.
  - ESR.STF_ERR.
  - ESR.BIT0_ERR.
  - ESR.BIT1_ERR.
  - CRCR.

  - FDCRC.
  - Error handling Registers(ERRIAR , ERRIDPR , RERRAR , RERRDR , RERRSYNR , ERRSR).

- The following CAN Protocol functionality will not be implemented.
  - BIT errors.
  - STUFF error.
  - CRC error.
  - FORM error.
  - ACK error.
  - OVERLOAD frames are not sent
  - ERROR frames are not sent
  - Detection and Correction of Memory Errors
- BTE feature will not be implemented.

25.4 Assumptions

The following assumption is taken while modelling the FlexCAN module:

- The registers and memories will be accessed as little endian.
- Bit interpretation of CAN Protocol can be abstracted for TLM use case. The dominant/recessive bit interpretation will not be modeled. Data will be sent as is.
- There are no bit errors during transmission, so CRC can be abstracted for TLM use case. CRC will not be implemented.
- Inter-frame space cannot be enforced when using LT with temporal decoupling coding style. Inter-frame time will be modeled by increasing the annotated time by that many bit times.
- Bit stuffing is a mechanism to reduce the transmission of too many similar bits in sequence to make sampling easier at receiving side. This can be abstracted for TLM use case. Bit stuffing will not be implemented.
- CAN is a serial protocol implemented using bit transfers over a single wire. The bus being a single wire which connects to multiple devices (using daisy chain topology) makes some bus functionalities to be implicit. For example, multi-casting of frames, bus monitoring at the transmitter end so on. For TLM use case then CAN RX/TX will be modeled as target and initiator socket.
- If CAN-FD frame is received, but FDEN bit is not set, frame is discarded.

Chapter 26
Flexible I/O (FlexIO)

26.1 Introduction

Flexible I/O (FlexIO) is a highly configurable module providing a wide range of functionality such as emulation of a variety of serial protocols like UART, I2C, SPI, SAI. FlexIO has flexible 16-bit timers with support for various trigger, reset, enable, and disable conditions.

26.2 Supported Features

The following features of FlexIO are supported:

- Array of 32-bit shift registers with transmit, receive, data match, logic, and state modes:
  - Double-buffered shifter operation for continuous data transfer.
  - 1, 2, 4, 8, 16, or 32 multi-bit shift widths for parallel interface support.
  - Interrupt, DMA, or polled transmit and receive operation.
- Highly flexible 16-bit timers with support for various internal or external triggers, reset, enable, and disable conditions:
  - Programmable baud rates independent of bus clock frequency.
  - Programmable logic mode for integrating external digital logic functions on-chip, or combining pin, shifter, or timer functions to generate complex outputs.
  - Programmable state machine for offloading basic system control functions from CPU, with support for up to eight states, eight outputs, and three selectable inputs per state.
- Integrated general-purpose I/O registers and pin rising or falling edge interrupts to simplify software.

26.3 Features Not Supported

The model FlexIO does not support the following features:

- Shifter concatenation to support large transfer sizes.
- Automatic start and stop bit generation.
- TLM serial protocol support, including but not limited to:
  - UART
  - I2C
  - SPI
  - I2S
  - Camera IF
  - Motorola 68K or Intel 8080 bus

26.4 Assumptions

The following assumptions were made:

- When a receive or match shift register is configured to both shift and store on the same cycle, then Shift register stores the post-shift register state.
- ONETIM condition only used for timer enable event.
- TIMCMP shall be configured with number of shift clock edges required for shifting data in or out based on TIMPOL configuration.

Chapter 27
Free-Running Oscillator Clock Generator (FRO)

27.1 Introduction

FRO_TUNER works as a trimmed range (200 to 400 MHz) clock source. Other frequencies may be available through post dividers outside of the module. It is capable of auto-trim relative to the XTAL clock.

27.2 Supported Features

The FRO block supports the following functionalities:

- All registers are modelled with access through target socket.
- FRO output clocks along with dividers and FRO 8Mhz output clocks are available as an output.
- TUNE_ERR, LOL_ERR, TRUPREQ are modelled through SCML commands.
- TUNEONCE_DONE is modelled through SCML command.
- Individual clock gating through CSR.CLKGATE bitfield is modelled.
- Disabling of FRO output clock using CSR.SMODE is modelled.
- TRIM_LOCK functionality is modelled through an SCML parameter. When both TREN and TRUPEN are 1, TRIM_LOCK is set after TRIM_TIME SCML parameter.
- Disabling of FRO output clock using FRO_PD and FROEN is modelled.

27.3 Features Not Supported

The FRO block does not support the following functionalities:

- Open loop mode and closed loop mode is not modelled.
- Functionality of registers related to TRIM FROTRIM, TRIMCNT, AUTOTRIM, TEXPCNT is not modelled.
- Functionality of CNFG2 and CNFG3.TRIMn_DELAY is not modelled.
- Functionality related to reference clock, REFCLKCNT and REFDIV of CNFG1 is not modelled. There is no reference clock that is input to FRO. Input frequency is set through SCML parameter.
- Fast startup option is not modelled.

27.4 SCML Commands

SCML Command Name	Description
set_LOL_ERR	Used to set LOL_ERR (Loss of Lock Error) through this scml command. No extra argument is required to be passed.
set_TUNE_ERR	Used to set TUNE_ERR through this scml command. No extra argument is required to be passed.
set_TRUPREQ	Used to set TRUPREQ (Trim Update Request Flag) through this scml command. No extra argument is required to be passed.
set_TUNEONCE_-DONE	Used to set TUNEONCE_DONE (Trim Update Request Flag) through this scml command after TREN and TRUPEN are both 1 and TUNEONCE is also 1. No extra argument is required to be passed.

Chapter 28
General Purpose Registers 0 (GPR0)

28.1 Introduction

This module specifies the general-purpose registers utilized for input and output pins.

28.2 Supported Features

The GPR_0 module implements the following features:

- GPR_0 is a register stub model.

28.3 Features Not Supported

The following features are abstracted out or not implemented in GPR_0 module:

- Updating status of current injection protection.
- Enable routing to fault channel A and B is not supported.
- Alt_hard_en of NETC ETHx tester trigger expose.
- Alt_hard_en of IPT tester trigger.
- The registers GPR_RWF_0/1/2/3/4/5/6, GPR_ROF_0/1 are not modelled.
- MUX selection between SDHC and XSPI loopback PAD.
- Enabling and disabling SDHC loopback pad electrical characteristics drivers.

Chapter 29
General Purpose Registers 1 (GPR1)

29.1 Introduction

This module specifies the general-purpose registers utilized for input and output pins.

29.2 Supported Features

The GPR_1 module implements the following features:

- CM7 SW Fault is supported.
- CM7 transition to quiescent state is supported
- Neutron clock gating is supported.
- Neutron reset is supported.
- Enabling/disabling access to CM7 TCM backdoor path is supported.
- Lock Enable for registers is supported.

29.3 Features Not Supported

The following features are abstracted out or not implemented in GPR_1 module:

- R52 Debug Error Event is not supported.
- Fault acknowledgement generation on peripheral_XBAR, AXBS XBAR, TCM XBAR
- Enable routing to fault channel A and Channel B is not modelled
- GPR_RWD_0 and GPR_RWD_3 register is not modelled.
- Debug functionality for CM7_0/1/2/3, DSPV, CSSI, CM4, Cortex-R52_0/1 is not supported.
- Updating status of current injection protection is not supported.
- ADC interleaving channel selection is not supported.
- Fence and Drain signal request is not supported.
- CAN timestamping feature is not supported.
- Blocking logic to gate reads and writes from PD1 domain to PD2 domain is not supported.
- Master idle indicator is not supported.
- RXLUT status registers (GPR_W1CD_10 – GPR_W1CD_16) are not modelled
- Time base enable is not supported.

Chapter 30
GPR_LPE_0

30.1 Supported Features

The model GPR_LPE_0 supports the following features:

- Enabling for destructive sources for escalation is supported.
- CM cores clock gating is supported.
- Gating of PAD to RGM is modelled.
- LCU and LPE CLK MUX selection is modelled.
- LCU IN mux selection is modelled.
- POR_WDOG is modelled
- LOCK Enable is modelled.

30.2 Features Not Supported

The model GPR_LPE_0 does not support the following features:

- Low power expose is not modelled.
  - Expose sheet is missing from the specification sheet.
- Alt hard enable for low power pads is not modelled.
  - Not enough information in specification sheet.
- Bypassing functional exit/entry proc is not modelled.
- LPE_CLKOUT/CLKOUT_STANDBY is not modelled due to missing information.
- GPR_ROD registers related to Injection protection bits is not modelled.
- EXTWAKE in GPR_RWF_0 is not modelled
- Skip application firmware authentication is not modelled.

Chapter 31
GPR_LPE_1

Supported Features
The model GPR_LPE_1 supports the following features:

- LCU clock muxing is supported.

31.1 Features Not Supported

The model GPR_LPE_1 does not support the following features:

- Enable fault routing to channel A and B.
- QREQ bit enable pin signal for LPUART,LPSPI, LPI2C.
- CAN timestamp feature is not supported
- Fault acknowledgment generation on LPE_XBAR.
- Stop1 and Stop2 mode trigger is not supported.

Chapter 32
Interrupt Controller (INTC)

32.1 Introduction

INTC schedules priority-based preemptive scheduling of interrupt requests (IRQs).

32.2 Supported Features

The following features of INTC are supported:

- All registers modeled.
- Generic slave interface to access registers modeled using tlm::tlm_target_socket<8>.
- Reset support implemented.
- 1024 interrupts supported:
  - 32 Software-Initiated IRQs through SSCIR0.
  - 992 Hardware Interrupts either through external interface or Software Trigger.
- Only in Hardware Vector mode, INTC sends the vector offset directly to the processor (4 bytes for each IRQ).
- Hardware and Software Vector Mode supported.
- Priority Management:
  - 64 unique priorities.
  - Pre-emption of lower priority.
  - Pre-empted priority stacking in Priority LIFO.
- Three clock cycle latency for hardware interrupt and four clock cycles for software interrupt supported.

32.3 Features Not Supported

The model INTC does not support the following features:

- In case of hardware vector mode, if reception of acknowledge signal and writing of EOIR register happens at the same time, then former wants to push into the LIFO and the latter wants to pop from the LIFO. This process is handled based on the system events callback sequence triggering. This sequence control is not handled.

32.4 Assumptions

- Number of Software-Initiated IRQs = 32.
- If high priority interrupt B occurs before interrupt A, it can be asserted to the processor and interrupt A becomes inactive. Interrupt A is not selected and asserted to the processor after processor signals End of Interrupt for interrupt B.

Chapter 33
Local Self-Test Control Unit (LSTCU)

Supported Features
The model LSTCU supports the following features:

- All registers are modeled.
- Generic slave interface to access registers are modeled using tlm::tlm_target_socket<32>.
- Reset support is implemented.
- LBIST sequential/concurrent execution.
- MBIST sequential/concurrent execution.
- Programmable LBIST execution time.
- Configuring MBIST execution time using SCML command.
- Configuring LBIST execution time using SCML command.
- Status and end flags to signal BIST completion.
- LSTCU error UF/RF lines to CSTCU.
- BIST failure supported through SCML command.

33.1 Features Not Supported

The model LSTCU does not support the following features:

- Isolation, reset and indication signals not modeled.
- ISODUR, PH1_DUR, SRCENDUR registers related functionality not modeled.

33.2 SCML Commands

The table below describes the SCML parameters for the LSTCU model:

Command Name	Description
set_mbist_runtime_ns	This command is used to set the time required by a MBST to complete its execution. Argument 1 specifies the MBIST number whose execution time is to be set and argument 2 specifies the time in nano seconds.

set_bist_status	This command is used to set the status of BIST to pass/fail. The first argument specifies BIST is MBIST/LBIST, second argument specifies BIST number and the third argument specifies BIST will PASS/FAIL.
set_fbist_invalid_ptr	This command is used for injecting valid or invalid pointer error in FBIST mode.
Set_lbist_runtime_ns	This command is used to set the time required by a LBST to complete its execution. Argument 1 specifies the LBIST number whose execution time is to be set and argument 2 specifies the time in nano seconds.

Chapter 34
Logic Control Unit (LCU)

34.1 Introduction

Logic Control Unit (LCU) selects multiple inputs from timers, Pulse Width Modulation (PWM) signals, and Input/Output (I/O) pads, and combines them using a programmable logic function to create output waveforms.

34.2 Supported Features

The LCU block implements the following functionalities:

- Upto three LCs with programmable logic function for generating output results.
- 12 inputs to and 12 outputs from each LC.
- Muxing to map any input to any LC.
- Independent filters for the rising and falling output states of each LC.
- Software override logic for inputs using multiple modes.
- Three force inputs with programmable edge filtering to force output states using multiple modes.
- Two sync inputs to control transition timing.
- Lock bit to block writes to configuration registers.
- Interrupt request generation based on output change or force event.
- DMA transfer request generation based on output change or force event.

34.3 Assumptions

The following assumptions were taken while modeling this peripheral:

- Configurations of the register does not trigger any output changes. The outputs change only when inputs are changed with prior configurations of registers.
- For rise and fall edge delay function, input is considered as output calculated from the LUT block. Output of LUT block changes instantaneously on changes in the inputs. Based on the configurations of the LCn_-FILT, output of the LUT block is delayed by n cycles. LCOUT gets updated after delay has elapsed which in turn, changes the output after calculations.
- Clearing of LCU_STS. FORCESTS, FORCESTS.FORCESTS happens immediately. There is no time delay. If force signal is cleared, then to clear the force status 1 (W1C) must be written.

Chapter 35
Low Power Comparator (LPCMP)

35.1 Supported Features

The model LPCMP supports the following features:

- Includes two 8-to-1 channel MUXes to select input signal from eight channels.
- Supports multiple operation modes to produce a wide range of outputs such as:
  - Sampled.
  - Windowed, which is ideal for certain PWM zero-crossing-detection applications.
  - Digitally filtered.
- Provides the following advance features for window and sample:
  - Window and sample signals can be inverted.
  - CMPO rising, falling or both edges closes the window.
  - CMPO level can be defined when window is closed.
- Provides selectable performance levels:
  - Low-Power (speed) mode.
  - High-Power (speed) mode.
- Provides a selectable inversion on comparator output.
- Provides interrupt and DMA support.
- Supports Round Robin Trigger mode.
- Includes an 8-bit resolution DAC.
- Provides a selectable supply reference source for DAC.

35.2 Features Not Supported

The model LPCMP does not support the following features:

- Supports programmable hysteresis control.
- Uses an external hysteresis at the same time the output filter is used for internal functions.
- Comparing consecutive CCR1[FILT_CNT] samples in filtering is not present.

35.3 Assumptions and Dependencies

- In RR trigger mode, RR_CH1OUT is updated at the same time when comparator result is calculated.

Chapter 36
Low Power Inter-Integrated Circuit (LPI2C)

36.1 Introduction

The LPI2C is a low power Inter-Integrated Circuit (I2C) module that supports an efficient interface to an I2C bus as a initiator and/or as a target.

36.2 Supported Features

The LPI2C block implements the following functionalities:

- Initiator based:
  - Command/transmit FIFO of 16words.
  - Receive FIFO of 16 words.
  - Command FIFO waits for idle I2C bus before initiating transfer.
  - Command FIFO can initiate (repeated) START and STOP conditions and one or more initiator-receiver transfer.
  - STOP condition can be generated from command FIFO, or generated automatically when the transmit FIFO is empty.
  - Host request input to control the start time of an I2C bus transfer.
  - Flexible receive data match can generate interrupt on data match and/or discard unwanted data.
  - Flag and optional interrupt to signal Repeated START condition, STOP condition, loss of arbitration (through SCML commands), unexpected NACK, and command word errors.
  - Supports configurable bus idle timeout and pin-stuck-low timeout (through SCML commands).
  - Supports circular FIFO.
- Target Based:
  - Separate I2C target registers to minimize software overhead because of initiator/target switching.
  - Support for 7-bit or 10-bit addressing, address range, SMBus alert and general call address.
  - Transmit data register that supports interrupt or DMA requests.
  - Receive data register that supports interrupt or DMA requests.
  - Software-controllable ACK or NACK.
  - Flag and optional interrupt at end of packet, STOP condition, or bit error detection (through SCML commands).

36.3 Features Not Supported

The LPI2C block implements the following functionalities:

- Pin Configuration - a TLM model and does not consist of SCL/SDA, and so on pins.
- Any delays in registers' read/write access.
- Multi initiator, clock stretching, and arbitration Glitch filter.
- Timing parameters.

36.4 Assumptions

The following assumptions are taken while modeling this peripheral:

- Host request trigger and input trigger are required to generate START condition, only when HREN bit in MCFGR0 register is enabled.
- Command 0x7 in MTDR register has no use and thus the model does not perform any transaction for this command.

36.5 SCML Parameters

The following SCML parameters are implemented for this model.

Identifier	Type	Default Value	Description
NUM_TX_FIFOS	Uint16_t	16	Depth of Transmit/Command FIFO.
NUM_RX_FIFOS	Uint32_t	16	Dept of Receive FIFO.
VERSOIN_ID	Uint32_t	0x01010003	Contains version numbers for the module design and feature set.

36.6 SCML Commands

The following SCML commands are implemented for this model.

SCML Command	Usage	Description
inject_arbitration_lost_error	To inject arbitration lost error.	Initiator has lost arbitration.
inject_pin_low_timeout_error	To inject Pin low timeout.	Pin low timeout has occurred.

inject_bit_error	To inject bit error.	Target has detected a bit error.

Chapter 37
Low Power Serial Peripheral Interface (LPSPI)

37.1 Introduction

The LPSPI is a Low Power Serial Peripheral Interface.

37.2 Supported Features

The LPSPI block implement the following functionalities:

- Requires minimal CPU overhead, with DMA offloading of FIFO register accesses.
- Supports DMA accesses and generates DMA requests.
- 32-bit word size.
- Configurable clock polarity and phase.
- Initiator mode-supports 8 peripheral chip selects.
- Target mode.
- 64-word transmit and command FIFO.
- 64-word receive FIFO.
- Flexible timing parameters in initiator mode, including SCK frequency and duty cycle, and delays between PCS and SCK edges.
- Continuous transfer option to keep PCS asserted across multiple frames.
- Full-duplex transfers support 1-bit transmit and receive on each clock edge.
- Half-duplex transfers support:
  - 1-bit transmit or receive on each clock edge.
  - 2-bit transmit or receive on each clock edge.
  - 4-bit transmit or receive on each clock edge.

37.3 Features Not Supported

The LPSPI block does not implement the following functionalities:

- SPI pins are abstracted as TLM FT SPI socket.
- Flexible timing parameters in initiator mode, including SCK frequency, and delays between PCS and SCK edges.
- Clock loopback.
- Configurable clock polarity and clock phase.

37.4 Assumptions

The following assumptions are taken while modeling this peripheral:

- Statement from the specification- The LPSPI initiates a SPI bus transfer when:
  - Data is written to the transmit FIFO – it is assumed here, first data entry push to the FIFO.
  - and the LPSPI is enabled.
- In case of full duplex communication, if TXFIFO of the target is empty, then LPSPI target transmits data configured as TX_DEFAULT_VAL parameter.

Chapter 38
Low-Power Universal Asynchronous Receiver/Transmitter (LPUART)

38.1 Introduction

The LPUART supports full-duplex, asynchronous, NRZ serial communication and comprises a baud rate generator, transmitter, and receiver block. The transmitter and receiver operate independently, although they use the same baud rate generator. The following describes each of the blocks of the LPUART.

38.2 Supported Features

The LPUART block supports the following functionalities:

- All registers modeled.
- Generic slave interface to access registers modeled using scml2::ft_target_socket<32>.
- SCML command has been implemented for noise error, parity error and frame error flag setting.
- Programmable baud rates (13-bit modulo divider).
- Transmit and receive baud rate can operate asynchronous to the bus clock:
  - Baud rate can be configured independently of the bus clock frequency.
  - Supports operation in stop modes.
- Interrupt, DMA or polled operation:
  - Transmit data register empty and transmission complete.
  - Receive data register full.
  - Receive overrun, parity error, framing error, and noise error.
  - Idle receiver detect.
  - Active edge on receive pin.
  - Break detect supporting LIN.
  - Receive data match.
- Programmable 7-bit, 8-bit, 9-bit or 10-bit character length.
- Three receiver wakeup methods:
  - Idle line wakeup.
  - Address mark wakeup.
  - Receive data match.
- Optional 13-bit break character generation / 11-bit break character detection.

- Configurable idle length detection supporting 1, 2, 4, 8, 16, 32, 64, or 128 idle characters.
- Selectable transmitter output and receiver input polarity.
- Hardware flow control support for request to send (RTS) and clear to send (CTS) signals.
- Independent FIFO structure for transmit and receive.
  - Separate configurable watermark for receive and transmit requests.
  - Option for receiver to assert request after a configurable number of idle characters if receive FIFO is not empty.

38.3 Features Not Supported

The LPUART block does not support the following functionalities:

- Infrared decoder.
- Programmable 2-bit stop bits – LIN RO does not support this feature.
- Parity feature is abstracted out in the model. Testing for parity error is supported via SCML command.
- Full-duplex UART communication.
- Doze mode.
- Receive and transmit timeout functionality.

38.4 Assumptions

- p_cts_b_in and p_rts_b_out are active low signals.
- Input and output triggers are separate pins.
- IDLE flag is set immediately after a character is discarded in case of match/wakeup operations. CTRL.IDLECFG is not taken into consideration.

Chapter 39
Message Interrupt (MSGINTR)

39.1 Introduction

The purpose of MSGINTR model is to convert message based write interrupts into wired interrupts..

39.2 Supported Features

The MSGINTR module implements the following features:

- Conversion of memory write transaction to interrupt event.
- Write to MSIIRn generates an interrupt and set associated with in MSIR.
- Read from MSIRn clears register and clears interrupt.

39.3 Assumptions

- It is assumed that when reading MSIRn register, it'll return the actual value and internally the bits will be cleared. Any subsequent read will return zero.
- At POR, the interrupts will be clear.

Chapter 40
Message Receive Unit (MRU)

40.1 Introduction

MRU couples with a module or processor. It receives messages for that module or processor from senders. The senders are other modules or processors.

40.2 Supported Features

The MRU block supports the following functionalities:

- AHB Slave interface implemented as TLM2.0 target socket.
- Upto six messaging channels. Each has its own set of registers.
- Upto four mailbox registers per channel.
- All registers are supported.
- Up to four hardware interrupts per channel which can be chosen optionally.
- Each channel has a channel enable, which allows or prevents writing to that channel's configuration, status, and mailbox registers.
- Variable-length messages with configurable status bit for each mailbox register.
- Channel register software reset.

Chapter 41
Messaging Unit (MU)

41.1 Introduction

The Messaging Unit module enables two processors within the SoC to communicate and coordinate by passing messages (for example, data, status, and control) through its interface. It also provides the ability for one processor to signal the other processor using interrupts.

41.2 Supported Features

The MU block supports the following functionalities:

- Memory-mapped registers:
  - The MU is connected as a peripheral under the peripheral bus on processor A-side and processor B-side.
- Synchronized message transfers between cores:
  - For sending data or messages from one MU-side to the other MU-side, the MU provides 4 transmit registers and 4 receive registers on each side of the MU.
  - The transfer of data messages between cores uses transmit empty and receive full flags provided on both sides of the MU.
  - The update of these transmit and receive flags is accomplished using a synchronization mechanism. There is inherent latency between updating the flag on one side and reflecting its status on other side.
- The MU has a 3-bit Flag Data register, which can be used to send flag data between two MU sides.
- Interprocessor Interrupts:
  - The MU has 12 interrupt sources on each side (Processor A-side, Processor B-side) that are used for signalling the other processor. The interrupts can be used for notification of Rx/Tx events and general-purpose signalling between the processors. Those interrupts can also be used for waking up the other core from low-power mode.

Each core can give a non-maskable interrupt to the other core.

- Monitor Core Operation mode:
  - The MU can monitor Core Operation mode including Low Power mode (RUN, HALT, WAIT, STOP, Power-Down) and reset state (in/out of reset) and save to status register. The corresponding interrupt can be generated and sent to another core.
- Flexible Processor configuration:

  - The processor can issue a programmed hardware reset to the other Processor by writing 1 to the HR bit in the Core Control Register 0 (CCR0).
  - The boot source for processor can be configured with the BOOT bits in the CCR0 register.
  - The core can be hold in reset state by configuring the RSTH bit in the CCR0 register.
  - The processor platform clock can be enabled to continue running when processor enters Stop mode, until the other processor also enters Stop mode. This allows processor to continue accessing peripherals on the other processor's bus even when it has entered a Stop mode.

- MU reset:
  - The processor can issue a reset to the entire MU, using a control bit (MUR) in any processor Control Register (CR).
  - The MUR bit is a self-clearing bit.

41.3 Features Not Supported

The MU block does not implement the following functionalities:

- BOOT bit does not have any functionality.

Chapter 42
Miscellaneous Control Module (MCM)

42.1 Introduction

The Miscellaneous Control Module (MCM) provides miscellaneous control functions and contains Cortex-M7 local memory descriptors. The purpose of the Model is to provide information.

42.2 Supported Features

The MCM includes the following features:

- Floating Point Exception monitor and interrupt control

42.3 Features Not Supported

- Program-visible information on the platform configuration and revision
- Write Abort Interrupt is not supported
- Local memory descriptors:
- ITCM
- D0TCM
- D1TCM
- ICACHE
- DCACHE

Chapter 43 Miscellaneous System Control Module (MS_CM)

43.1 Introduction

The Miscellaneous System Control Module (MSCM) contains CPU configuration registers and on-chip memory controller registers.

43.2 Supported Features

The MSCM module implements the following features:

- Software-accessible processor core configuration information (CPU configuration registers).
- Interrupt router control registers.
- Message-based interrupt configuration and management registers.

43.3 Features Not Supported

The following features are abstracted out or not implemented in MSCM module:

- Functionality for below registers is abstracted.

Address Offset	Register
500h	Memory Execution Control (XN_CTRL).
600h	Enable Interconnect Error Detection (ENEDC).
604h	Enable Interconnect Error Detection (ENEDC1).
700h	AHB Gasket Configuration (IAHBCFGREG).

43.4 Assumptions

- In register memory execution control [XN_CTRL].

- XN_CTRL.SLK
  - While clearing SLK bit [soft lock clear], other bitfields also get written in same write transaction.

Chapter 44
Mode Entry Module (MC_ME)

44.1 Introduction

The MC_ME controls the SoC mode and mode transition sequences in all functional states. Its features are:

- It defines the device mode and controls its configuration.
- Triggers reset and standby entry sequence.
- Trigger hardware processes in partition.
- It controls state of IP blocks in partition and captures its status (active/inactive).
- It also displays the clock status of the attached IP blocks.
- It controls attached core clock (enable/disable) and captures its status.

44.2 Supported Features

- Configurable modes of operation by software. Modes implemented are:

RESET	STANDBY	RUN

- Enable/disable control of connected IP Blocks in partition.
- Clock control of cores in partition.
- Hardware processes control in partition.

44.3 Assumptions

- MC_ME enters active mode called RUN mode coming out of reset and remains in it till mode transition takes place.
- When Standby Mode is in progress (that is, mode configuration register is programmed with STANDBY enable and mode update register is set to allow mode change) MC_ME waits for main core to get in WFI then:
  - If RESET mode transition is requested or p_rst_in signal is asserted, the standby in progress will get cancelled and no standby request will be sent to MC_PCU/MC_RGM.

  - If Main CoreID register values are updated, then a warning will be thrown to indicate that software is trying to write to this register when standby is in progress.
  - If again standby mode transition is requested by programing Mode configuration register then the request will have no impact.

- No configuration information is send to RGM through p_me_bus_out or received from RGM through p_-me_bus_in as the specification does not specify what to send/receive.
- No functionality of p_exit_from_standby_in port functionality is implemented due to incomplete information.
- Standby exit sequence is not triggered through MC_ME as it requires clarification.

Chapter 45
MRAM Controller (MRAMC)

45.1 Introduction

The MRAM Controller (MRAMC) IP is used to connect to MRAM to the system. It acts as a bridge IP that converts system transactions to MRAM interface transactions.

45.2 Supported Features

The following features of MRAMC are supported:

- Supports Single, Quad, and Octa pages write.
- Restriction on number of pages that can be written to the specific memory space.

45.3 Features Not Supported

The model MRAMC does not support the following features:

- UTest Mode:
  - It is not implemented functionally.
- Prefetch and Read Buffering:
  - Can be abstracted out in LT model. Does not impact functionality.
- Error Status Register (ERRSTAT0):
  - ECC errors can be set using SCML commands.
- Safety Mechanism:
  - No DED or QED ECC implementation.
- Error Injection Control:
  - Safety logic/mechanism is not implemented.

45.4 Assumptions

The following assumptions were made:

- When SMW made as 1, WDATA, WSTRB, WDLR, NUM_WP or any other required field shall not be updated.
- If write is in progress, new address will only be updated on WADDR once current transaction is completed for respective active channel.
- F16ASM LOCK0 and LOCK1 will not reflect default lock status as reset value is 0x00000000 until there is any write on MRAMC lock registers.

45.5 SCML Commands

The following parameters are provided as SCML Commands to set respective errors:

Name	Attributes (Argument = “Address”)	Description
setMEEE	Arguments requires = 0	Sets MRAM EDC after ECC error.
setMCE	Arguments requires = 1	Sets MRAM Controller Error.
setDEE	Arguments requires = 1	Sets Data ECC error.
setCEE	Arguments requires = 1	Sets Control ECC error.
setCPE	Arguments requires = 1	Sets Control parity error.
setPWE	Arguments requires = 1	Sets Posted write error.
setEER	Arguments requires = 1	Sets EER.
setTBC	Arguments requires = 1	Sets TBC.
setDBC	Arguments requires = 1	Sets DBC.
setSBC	Arguments requires = 1	Sets SBC.
setAEE	Arguments requires = 0	Sets AEE.
setEEE	Arguments requires = 0	Sets EEE.
setWCE	Arguments requires = 1	Sets WCE.
setUE	Arguments requires = 0	Sets UE.
setWAE	Arguments requires = 0	Sets WAE.
setWTE	Arguments requires = 0	Sets WTE.

Chapter 46
Periodic Interrupt Timer (PIT)

46.1 Introduction

The functionality of the PIT module can be used to generate interrupt as well as triggers. PIT has 4 timers. PIT has chaining mode support to implement a 64 bit timer.

46.2 Supported Features

The PIT block supports the following functionalities:

- 4 32-bit countdown timers
- Independent timeout periods for each timer
- Ability to chain two timers into a 64-bit lifetimer
- Ability of timers to generate interrupts
- Ability of timers to generate trigger pulses
- Maskable interrupts
- One RTI countdown timer to wake up the CPU
- Option to generate the RTI even when the bus clock is off
- Power saving with a separate input clock for the RTI timer. All other timers share a common core clock.

46.3 Features Not Supported

The PIT block does not support the following functionalities:

- Debug mode (FRZ bit in MCR register).

46.4 Assumptions

- DMA Trigger will be asserted unconditionally for 1 clock cycle when counter value underflows.
- Timer 0 also cannot be chained.

Chapter 47
Power Control Unit (MC_PCU)

47.1 Introduction

The power control unit (MC_PCU) is used for initiating a power mode entry/exit sequence. The support for multiple power modes helps reduce the overall chip power consumption. Power can be saved by disconnecting parts of the chip from the power supply. The blocks inside the chip are grouped into multiple parts having this capability, which are called "power domains".
Supported Features
The following features of MC_PCU are supported:

- Finite state machine to handle moving in and out of the STANDBY, LP, RUN mode.
- Seven states implemented are higher abstraction –
  - IDLE - fsm is in idle and waiting for standby event.
  - StandbyEntry – includes all power down states.
  - StandbyWait – waiting for wakeup event or functional reset.
  - StandbyExit – include all power up states.
  - LPRunEntry– includes all power down states.
  - Wait_LP – waiting for wakeup event or functional reset.
  - LPRunExit – include all power up states.
- Standby entry request from RGM.
- Support for wakeup event / functional / destructive reset event during standby.
- Low Power request from RGM.
- Support for wakeup event / functional / destructive reset event during Low Power mode.

47.2 Features Not Supported

The model MC_PCU does not support the following features:

- Input/Output isolation control and status signals.
- Interface to the external PMIC.
- SIRC power down/up sequence.
- Reset Handling when coming out of Standby/Low Power is done in RGM not in PCU.

Chapter 48
Power Management Controller (PMC)

48.1 Introduction

The Power Management Controller (PMC) regulates and monitors power supplies of the respective power domains.
Supported Features
The following features of PMC are supported:

- Low-voltage and high-voltage detection (LVD and HVD) precision monitors as indicators for software.
- A low-voltage reset (LVR) precision monitor for all system-relevant power domains.
- Monitor Self-Test is implemented (MONSELF[SEL] = 1 to 20 only).

48.2 Features Not Supported

The model PMC does not support the following features:

- PMC will not implement PGOOD signal functionality.
- Go/no-Go Sensor (GnG) is abstracted out.
- CONFIG[LVRAREC] and CONFIG[VDDRAMEN] are not implemented.
- PMC will not monitor below signals:

External power supply pin	I/O	Description
V08_PD0_CAP	Supply output	V08_PD0_CAP is the core and logic supply for the low-power domain, also called power domain 0 (PD0). A linear low-power regulator inside PMC drives this output.
V08_PD1_CAP	Supply output	V08_PD1_CAP is the core and logic supply for the medium power domain (one CPU only, also called PD1 for power domain 1). A linear regulator inside PMC drives this supply.

V18_PD0_CAP	Supply output	V18_PD0_CAP is the analog supply for the standby domain. A linear low-power regulator inside PMC drives this supply.
V18_PD12_CAP	Supply output	V18_PD12_CAP is the analog supply for the full performance and low power run domains (PD2 and PD1). A linear regulator inside PMC drives this supply.
V18_PD12B/C/D/E	Supply output	On the package, several power rails share V18_PD12: V18_1B, V18_1C, V18_1D, and V18_1E. They supply various peripherals, and PMC implements a go/no-go monitor for each of them.
VIOREF	Supply output	VIOREF is derived from VDD_-HV_A inside PMC (divided by two). It is an intermediate voltage level, required to support FET cascodes in the VDD_-HV_A domain.
VRC_CTRL	Output	If this option generates V08_-PD2, VRC_CTRL connects to the base of an external BJT.

48.3 Assumptions

The following assumptions were made:

- System is reset when VDD_HV_A, V08_PD2 is less than respective voltage threshold.
- System reset is not asserted if Self-Test is enabled.

Chapter 49
Power-on Reset Generator (POR_GEN)

49.1 Introduction

POR_GEN is a standalone block that receives the reset sources and power domain status in the chip. It, then, combines them into the output power-on reset (POR) event.

49.2 Supported Features

The POR_GEN block supports the following functionalities:

- Generating POR output for each reset source and updating its flag.
- Min_pulse block is implemented as a delay on out signal.

Chapter 50
Real Time Clock (RTC)

50.1 Introduction

The Real-Time Clock (RTC) is a free-running counter used for time keeping applications. The RTC can be configured to generate an interrupt at a pre-defined interval independent of the mode of operation (run mode or low power mode). If in a low power mode, the RTC interval is reached, the RTC first generates a wakeup and then asserts the interrupt request. The RTC also supports an API function used to generate a periodic wakeup request to exit a low-power mode or an interrupt request.

50.2 Supported Features

The RTC model implements the following functionality:

- 32-bit counter.
- Selectable counter clock sources (IRCs and OSCs)
  - Clock source 0.
  - Clock source 1.
  - Clock source 2.
  - Clock source 3.
- Optional 512 prescalar and optional 32 prescalar connected in series in the clock path feeding the 32 bit counter.
- 32-bit counter supports times up to greater than 1.5 months with 1 ms resolution.
- 32-bit compare value to support interrupt intervals of 1 second up to greater than 1 h with 1 second resolution.
- RTC interrupt with interrupt enable.
- Counter runs in all modes of operation.
- RTC status and control register are reset only by POR.
- RTC counter is reset when the counter is disabled by software and by POR.
- Autonomous periodic interrupt support includes:
  - 32-bit compare value to support wakeup intervals of 1.0 ms to 1 second.
  - Wakeup logic has a separate enable to support changing compare value while RTC is running.
  - API interrupt with interrupt enable.
  - API compare value can be modified while RTC is running.
- Optional interrupt for RTC match, API match, and RTC rollover.

50.3 Features Not Supported

The RTC block will not implement the following functionality:

- Difference between counter clock and bus clock is not considered.
- Clock synchronization logic.
- RTCVAL synchronization with RTC clock.
- Debug mode will not be implemented.

50.4 Assumptions

- Prescaler enabling/disabling, while the RTC is running, is not supported.
- Clock source change while RTC is running, is not allowed.
- If clock period is updated while RTC is running, the changes will be done in following sequence-
- RTC counter is paused.
- New period is calculated and Prescaler is applied, if any.
- RTC counter is resumed from old value. (counter is not restarted).

Chapter 51
Reset Domain Controller (RDC)

51.1 Introduction

The Reset Domain Controller (RDC) controls the software resettable domains and allows the assertion/de-assertion of reset for them.

51.2 Supported Features

The following features of RDC are supported:

- Generates reset sequence for software resettable domains/partitions on writing of MC_RGM RSTn_0[PERIPH0_RST] bit for partition reset assertion/de-assertion.
- Provides the reset phase indicators.

51.3 Features Not Supported

The RDC block does not support the following functionalities:

- Triggers for the memory repair/trims loading.
- Generation of the punched-through clock enable signals.
- Enable/disable for X-bar interface.

51.4 Assumptions

The following assumptions are taken while modeling this peripheral:

- The RDC is a superset model and hence may contain additional registers/bitfields that are not present in all platforms (for example, S32S247TV), but may be needed for another platform (for example, S32G274A).
- If the control bit of the current phase or subsequent phases is asserted while state machine is running, then bit in only the control register is updated, but state machine will have no state transition.
- If all the control bits of prior phase are asserted, then state machine moves to the starting phase. For example, if the state machine is in phase 3 and RDn_CTRL_REG[PHASE2] and RDn_CTRL_REG[PHASE1] are asserted, then the state machine moves to phase 1.

Chapter 52
Reset Generation Module (MC_RGM)

52.1 Introduction

The Reset Generation Module (MC_RGM) centralizes the different reset sources and manages the reset sequence of the chip. It provides a register interface and the reset sequencer. There are various registers available in this module to monitor and control the chip reset sequence.

52.2 Supported Features

The MC_RGM block supports the following functionalities:

- Implement all POR, functional and destructive resets via Power on, External, Destructive and Functional input reset sources.
- Implements all the states of Reset State Machine.
- Signaling of reset events after each reset sequence (reset status flags).
- Alternate event generation / Conversion of reset events to interrupt request events.
- Functional and destructive reset escalations.
- Peripheral Reset Individually.
- MC_ME configuration via MC_ME interface.
- Bidirectional reset behavior configuration.
- Software configurable external reset assertion control.

52.3 Features Not Supported

The MC_RGM block does not support the following functionalities:

- Debug control reset not modeled.
- LPDEBUG register functionality not modelled.
- Test Mode Functionality not supported.
- JTAG initiated reset not modeled.
- Boot mode functionality not modeled.

52.4 Assumption

- Signals with naming scheme "_b" at the end have been assumed to be active low unless timing diagram says otherwise.
- Timing diagrams have been given priority in case of ambiguity in description.
- The register RGM_FREC is only resettable in destructive reset sequence.
- rst_periph_b[n] reset signals can be asserted independent of the Reset state machine.
- Signals output to RSL interface namely rst_por_b[1:0], rst_dest_b[1:0] , rst_func_b[1:0] & rst_func_-early_b[1:0] described in RTL IP specification signals description have parameterized width of NUM_-OF_PD.
- FIRC_STABLE will be set equal to the number clock cycles; taken after input "**por_b**" is de-asserted, until FIRC clock is stabilized.
- RGM_DRET.DRET field is set to Parameter DEST_ESC_DEF_VAL on Power On Reset. It is by default set to 0 to match the normal behavior.
- RGM_FRET.FRET field is set to Parameter FUNC_ESC_DEF_VAL on Power On Reset. It is by default set to 0xF to match the normal behavior.
- Individual peripheral support for reset is functionally modeled, the relevant PRST and PSTAT registers are also modeled.

Chapter 53
RX Lookup Accelerator (RXLUT)

53.1 Introduction

The main function of RXLUT is to return a value/decision corresponding to a frame ID and channel number to meet system performance by searching entries in a table. This assist in speeding the decision-making process on receive frames by saving huge number of cycles otherwise required for software search. It has a table containing entries in order of 1000 or more to search with three kinds of scenarios.

53.2 Supported Features

The RXLUT model implements the following functionality:

1. It searches a pre-populated lookup table corresponding to a frame ID and channel number combination, to find any match and present the result.
2. It supports access mode and search mode operation on entry table.
3. It allows read and write access for each entry in entry table.
4. It supports three type of entries in entry table:
   - Exact entry
   - Mask entry
   - Range entry
5. It supports flushing of all entries present in entry table.
6. It supports two types of search operation.
7. Normal search.
8. Priority search.
9. It supports multiple search enqueue till input command FIFO is not full in normal search.
10. It supports storing the result of search operation in output FIFO in normal search.
11. It generates interrupt when any match found for priority search or normal search operation.
12. Word read/write support on registers.
13. Number of masters supported 32.
14. Reset values for individual AIPS instances have been modeled.
15. Partition slots programmed for additional protection before VMID checks.

53.3 Features Not Supported

The RXLUT block does not implement the following functionality:

1. RXLUT CAN frame ID rearrangement feature not modeled.

Chapter 54
Semaphores2 (SEMA42)

54.1 Introduction

The hardware semaphore module is an architecture-neutral solution that provides hardware-enforced gates, as well as other useful system functions related to the gating mechanisms.

54.2 Supported Features

The SEMA4 module implements the following features:

- The model has a target interface to configure the internal registers.
- Module definition supporting 16 hardware-enforced gates in a multi-domain.
- The model supports locking/unlocking of a gate by same initiator.
- Secure reset mechanisms are supported to clear the contents of individual gates, as well as a clear all capability.

The Domain IDs for different masters as configured in /ECU/MCU/Base/Peripherals/System/SEMA4_0 are:

Domain Name	Physical Master ID
R52_0	0x80
R52_1	0x81
CM7_0	0x0
CM7_1	0x1
CM7_2	0xB
CM7_3	0xF
CM4_0	0xA

The Logical Domain ID for a particular core can be changed by the following command:
set_domain_id <physical master-id > <New logical domain-id>
For example, to assign a Logical Domain ID of 10 to M4 core:
set_domain_id 9 10

54.3 SCML Parameters

The following table describes the SCML parameters used by the SEMA4 module.

Name	Default Value	Description
USE_DID_FROM_XRDC	1	When this parameter is set to true, SEMA4 uses the DID (Domain ID) as set by XRDC for locking/unlocking of gates. Otherwise, the AMBA Master ID is used (as before).

Chapter 55
Sinc Filter (SINC)

55.1 Introduction

SINC converts an external ADC sigma-delta modulator bitstream to a data stream.
The converters are based on:

- A maximum of 3-order sinc digital decimation filters with FastSinc support.
- A maximum selectable oversampling ratio (OSR) of 2048.

55.2 Supported Features

The SINC block supports the following functionalities:

- Filter channels with dedicated modulator clocks, bitstreams, and triggers.
- Modulator clock outputs.
- One software and one hardware trigger per channel.
- One primary filter (PF) for each channel, with a shift and cascaded integrator comb (CIC) filter, for 24-bit precision.
- High-pass filters (HPF) for DC removal.
- A 16-entry FIFO with watermark support for each channel.
- Two DMA requests per channel with various event sources.
- Various interrupt sources available per channel.
- Programmable high and low detection limits.
- Support for short-circuit detector (SCD), clock-absence detector (CAD), and zero-cross detector (ZCD).
- Programmable shift operations.
- Programmable bias (offset) operations.

55.3 Features Not Supported

The SINC block does not implement the following functionality:

- Support for a Manchester-encoded bitstream is abstracted.
- Break signal outputs to turn off the external peripheral when a protection event occurs has not been implemented.
- Pulse-triggered output for additional control and monitoring is abstracted.

55.4 SCML COMMANDS

Scml_command	Description
Generate_output	Generates SINC output based on valid channel number

Chapter 56
Slow Crystal Oscillator Digital Controller (SXOSC)

56.1 Introduction

The SXOSC is used to control the functioning of the external crystal oscillator from the hardware perspective, from the VP perspective it generates free running output clock of frequency defined by SCML parameter. The model will have TLM2.0 FT target socket for accessing the memory-mapped registers. The model can be reset via reset pin.

56.2 Supported Features

Following is the list of features of **SXOSC** model:

- TLM2.0 FT socket is used for memory register access.
- Status register to provide the current operating state as stable/unstable.
- Default frequency is set to 32 KHz through SCML parameter.
- Reset pin is modelled.
- The SXOSC output clock stabilizing time will be:
- Stabilizing time (in ps) = EOCV * 128 * 4 * (clock_period in ps)

56.3 Features Not Supported

The SXOSC block does not support the following functionalities:

- External Crystal pins are abstracted.

56.4 Assumptions

- The default frequency of output clock is set to 32 KHz through SCML parameter and can be modified in future as per the need.
- The reset value of EOCV is 0x7D & maximum value of EOCV is assumed to be 0xFF.
- The output SXOSC clock will start and Status bit OSC_STAT will be set as soon as the counting till EOCV completes.
- Four times of the time period of the output clock is used for counting the stabilization time.

Chapter 57
Software Watchdog Timer (SWT)

57.1 Introduction

The model is a watchdog timer. This model needs to be serviced at specific intervals. If the software fails to service the timer then a system reset is generated.

57.2 Supported Features

The SWT model implements the following functionality:

- Slave interface asTLM2.0 target socket.
- Seven memory-mapped 32-bit registers.
- Programmable selection of system or oscillator clock, for timer operation
- Reset input for resetting the SWT internal registers and timer.
- Reset or interrupt generation on timeout.
- Fixed or keyed service operation for restarting the timer.
- Window mode or regular mode for servicing the timer.
- System reset or bus error generation on invalid accesses.
- Hard and soft lock for configuration registers.
- Master access protection enabling/disabling access to specific masters.

57.3 Features Not Supported

The SWT block will not implement the following functionality:

- Debug mode (SWT_CR.FRZ) not modeled.
- No time lag in recognition of service sequence or configuration changes would be modeled as the model follows LT style of modeling.

57.4 Assumptions

- Since the SWT follows LT style of modeling, there would be no delay in recognition of service sequence or configuration changes.
- In keyed service sequence mode (SWT_CR.SMD = 1), if the pseudo-random key is modified between the writes to SWT_SR register then the service sequence detection which began with the first write is invalidated. New service code sequence (based on the new value of SWT_SK register) needs to be written again.

Chapter 58
Synchronous Audio Interface (SAI)

58.1 Introduction

Serial Audio Interface (SAI) provides an interface that supports full-duplex serial digital audio interfaces with frame synchronization formats such as I2S, AC97, TDM, and Codec/DSP interfaces.
Supported Features
The following features of SAI are supported:

- Transmitter with independent bit clock supporting 4 data lines.
- Receiver with independent bit clock supporting 4 data lines.
- Maximum frame size of 16 words per data line.
- Word length of 8 to 32 bits.
- Word length configured separately for the first word and remaining words in a frame.
- Asynchronous 8 * 32-bit FIFO for each transmit and receive data line supports:
  - Graceful restart after FIFO error.
  - Automatic restart after FIFO error without software intervention.
  - 8-bit and 16-bit data packing into each 32-bit FIFO word.
  - Multiple-data-line FIFOs combining into single-data-line FIFO.
- Independent 32-bit timestamp counters and bit counters for monitoring transmit and receive progress.
- Single external bit clock interface for all TX channels (Assumption: BCD can define bclk direction for all TX channels, so all channels might use single external clock source).
- Transmission restarts from same word after a (word_width*bitclock_period) time if TX_FIFO is updated with word provided FCONT==1. During this period 0s are transmitted.

58.2 Features Not Supported

The model SAI does not support the following features:

- No debug mode is implemented. TCRS[DBGE] and RCSR[DBGE] is not modelled, or no functionality related to it is modelled.
- BCI (Bit clock Input) No pad output delay is implemented.
- BCP (Bit clock polarity) As data will be transmitted after n bit clocks, rising and falling edge is not considered where n is number of bits transmitted.
- No Frame Sync is implemented and related bitfields are not modeled except FSE where additional delay of 1 bit clock (along with 4 bits clocks frame sync delay) is added before start of Transmission or Reception once Transmitter or Receiver is enabled respectively.
- Channel Mode (CHMOD): When slots are masked always 0s are transmitted. 3-state output is not implemented that is, TDM mode.

58.3 Assumptions

The following assumptions were made:

- During FIFO combine mode, FIFO writes by SW are allowed only through TDR0.
- When FIFO packing is enabled, SW will update FBT as below:
  - 8-bit packing (LSB-First): 0b00000
  - 8-bit packing (MSB-First): 0b00111
  - 16-bit packing (LSB First): 0b00000
  - 16-bit packing (MSB First): 0b01111
- When FIFO packing is enabled, W0W shall be equal to WNW.
- When FIFO Combine mode enabled on FIFO writes, TDR0 shall be used to write data to respective enabled channels which will be updated in rotation. If FIFO Packing (8/16-bit) is enabled while FIFO combine mode enabled on FIFO writes, in 16-bit FIFO packing mode with N channels enabled, first packed word will be formed after 2*N 16-bit words write on channel 0. Similarly, it'll take 4*N 8-bit words write on channel 0 to have first 32-bit packed word on TXFIFO if 8-bit FIFO packing is enabled.
- When FIFO Combine mode enabled on FIFO read, if word is masked on respective channel, TXFIFO won't be read, and SAI will transmit 0. Channel pointer will be incremented to next channel to read next word. For example, we have enabled 4 channels (CH0, CH1, CH2 & CH3), and 2nd word is masked. First word will be read from CH0, Second word will be sent as 0 on CH1, Third word will be send from CH2, Fourth word will be sent from CH3 and channel pointer will reset back to CH0 and continue sending data in rotation.
- When word is masked FIFO pointer won't be incremented.

58.4 SCML Commands

The following parameters are provided as SCML Commands to set respective errors:

Name	Attributes (Argument = "Address")	Description
setTSEF	Arguments requires = 0	Sets TX Sync Error Flag
setRSEF	Arguments requires = 0	Sets Rx Sync Error Flag

Chapter 59
System Integration Unit Lite2 (SIUL2)

59.1 Introduction

The module is an SIUL2 device which is used for the management of pads and their configurations. It controls the multiplexing of the alternate functions used on all pads, as well as being responsible for the management of the external interrupts to the device.

59.2 Supported Features

The SIUL2 model implements the following functionalities:

- 364 I/O pads are supported along with input buffer enable, output buffer enable, and safe mode control functionality.
- Pin assignment function for both input and output pins are supported using MSCR and IMCR register.
- GPIO functionalities are supported using GPI, GPO, and associated registers.
- 16 external interrupts along with edge detection, independent mask, and independent DMA channel are supported.

59.3 Features Not Supported

The SIUL2 model does not implement the following functionalities:

- Functionalities corresponding to electrical characteristics of the pads including Edge Rate Control, Input level selection, pull down/up, injection current, drive strength, slew rate are not supported.
- Glitch filter and associated functionality is not supported.
- Safe mode is not supported in LT use-case.
- SST configuration registers is not supported yet.

59.4 Assumptions

The following assumptions are taken while modeling this peripheral:

- Ignoring entries from sheet, where direction and Mux number are not matching.
- As per instance, PGPDO and MGPDO are ignored for corresponding missing GPDO or MSCR registers.

Chapter 60
System RAM (SRAMC)

60.1 Introduction

The model acts as an interface between OCP system and System RAM. SRAMC initializes both banks of SRAM with double word value set by the parameter and reports any error, which occurs during initialization. SRAMC also provides status register which reports the initialization progress as well as different error cases for the whole SRAM system. This model also provides address associated with an ECC error during initialization.
There will be a single report for each error cycle for all the controller instances. Because only one location is reported, in case of multiple errors, it selects one over the others.

60.2 Supported Features

The SRAMC module implements the following features:

- SRAMCTL's memory-mapped registers are supported using scml2::register.
- TLM2 target port interface supported to access memory-mapped registers.
- RAM error injections feature supported using SCML2 command.
- RAM initialization supported using IPDONE and IPEND.
- RAMECCNT.ECCNT feature implemented.

60.3 Features Not Supported

The following features are not implemented in SRAMC module:

- Support for initialization of the SRAM arrays to a valid ECC state for each entry:
  - Initialization sequence with optional wait states, which can be used to limit power surge.
  - SRAM arrays are initialized in parallel.
  - Software programable portion of the SRAM for initialization.
- ECC Hsiao scheme for the RAM, which enables single-error correction, and double-error detection (SECDED).
- Support for an option to block access and suppress any error indicator for that access.
- Error injection on memory read data from an external source.
- Low-power control on memory cuts.
- Safety alarm and monitor for SRAMCTL pipelines.
- Access latency for pipelines .

60.4 Assumptions

The following assumption is taken while modelling the SRAMC module:
If SRAM initialization is triggered by software and during initialization if any neg edge occurs at p_sram_init_-in, it will cancel the running initialization. Similarly if SRAM initialization is triggered by p_sram_init_in and if RAMCR[INIT] is changed to zero during initialization, it will abort the initialization.

60.5 SCML Parameters

The following table describes the SCML parameters used by the SRAMC module.

Name	Default Value	Description
INIT_TIME	100	Initialization time(In MS) of SRAM.

60.6 SCML Commands

The following table describes the SCML commands implemented for SRAMC module.

SCML Commands	Description	Arguments
cmd_RAMSR	If this command is executed, SRAMC sets the error bits in status register and corresponding error signal is asserted. command processor parameter can be of following type. Error_type: SGLERR, MLTERR, ADR-ERR To set the EINOF and SYND, arg[0]->SYND {uint_8} and arg[1]->EINFO {unit_8}, To set the Error ("SGL_-ERR\",”MLT_ERR\","ADR_ERR") Example: send command like below (“Cmd_RAMSR”, “0x23”, “0x22”, “SGL_ERR”)	• Error_type ("SGL_ERR\",”MLT_-ERR\","ADR_ERR") • ECC Syndrome value(SYND) • EINFO
cmd_RAMSYSA	Set the value of the RAMSYSA register Example: (“cmd_RAMSYSA”, “0x123455”)	RAMSYSA.SYSA
cmd_RAMMEMA	To set or clear the RAMMEMA.BANK and RAMMEMA.MEMA 0-16 are accessible Example: (“cmd_RAMMEMA”, “0x12345678”)	cmd_RAMMEMA

Chapter 61
System Timer Module (STM)

61.1 Introduction

The System Timer Module (STM) is a 32-bit timer designed to support commonly required system and application software timing functions. The STM includes a 32-bit up counter and four 32-bit compare channels, with a separate interrupt source for each channel. The counter is driven by the system clock divided by an 8-bit prescale value (1 to 256).

61.2 Supported Features

The STM module implements the following features:

- 32-bit up counting timer.
- Enabling/disabling through STM_CR.TEN.
- 8-bit clock prescaler using STM_CR.CPS.
- Current counter value reading through STM_CNT.
- Four Compare channels.
- Compare value through STM_CMPn register.
- Enabling/disabling of compare channels through STM_CCRn.CEN.
- Status update through STM_CIRn.CIF and interrupt generation when counter reaches compare value.

61.3 Features Not Supported

The following features are abstracted out or not implemented in STM module:

- Timer stop/run in debugger mode using STM_CR.FRZ.
- Any changes in the clock source frequency are accounted for only after the STM timer overflows, or when STM_CNT or STM_CR registers are written (whichever is earlier).

Chapter 62
Temperatire Sensor Unit (TEMPSENSE)

62.1 Introduction

TEMPSENSE is a high-accuracy temperature sensor. It provides control for Single One-Shot, Continuous, and Periodic One-Shot.

62.2 Supported Features

The TEMPSENSE block supports the following functionalities:

- Provide a voltage proportional to the temperature which will be read out by ADC.

62.3 Features Not Supported

The TEMPSENSE block does not support the following functionalities:

- Continuous measurement mode is not supported for S32K5 platform.

Chapter 63
Trigger MUX (TRGMUX)

63.1 Introduction

The TRGMUX registers contain fields for selecting trigger sources for peripheral modules. TRGMUX registers can be written only in supervisor mode.

63.2 Supported Features

The TRGMUX module implements the following features:

- All registers for Trigger Multiplexor (TRGMUX) modeled.
- Register/memory interface implemented through scml2::ft_target_socket<32>.
- Number of trigger inputs configurable through SCML parameter.
- Number of trigger outputs configurable through SCML parameter.
- Input selection bit width will be configurable through SCML parameter.
- Reset pin implemented.
- All the individual unused trigger outputs configured through SCML parameter.

63.3 Assumptions

The following assumption is taken while modelling the TRGMUX module:
It is assumed that TRGMUX.LK bit can be written as 0 after reset multiple times but once 1 is written to LK bit, it cannot be written again (either 1 or 0) until next reset.

63.4 SCML Parameters

The following table describes the SCML parameters used by the TRGMUX module.

Name	Default Value	Description
NUM_INPUTS	36	Number of inputs.
NUM_OUTPUTS	18	Number of outputs
UNUSED_OUTPUTS	“ ”	Unused output ports (comma separated decimal values).

SEL_WIDTH	6	SEL Bit-field width. Need to be carefsul with width as 8 as in that case SEL3 will overwrite LK.

63.5 SCML Commands

The following table describes the SCML commands implemented for TRGMUX module.

SCML Commands	Description	Arguments
print_routing_table	This command prints the current Input to output mapping configured by software for a particular TRGMUX instance.	None

Chapter 64
Ultra Secured Digital Host Controller (uSDHC)

64.1 Introduction

uSDHC provides the interface between the host system and the eMMC, SD card, and SDIO. The module acts as a bridge, passing host bus transactions to the eMMC, SD card, and SDIO by sending commands and performing data accesses to/from the cards. It handles the SD card/SDIO/eMMC protocols at the transmission level.
Supported Features
The following features of uSDHC are supported:

- Conforms to the SD host controller standard specification version 3.0.
- Compatible with the MMC system specification version 4.2/4.3/4.4/4.41/4.5.
- Compatible with the SD memory card specification version 3.0 and supports the extended capacity SD memory card.
- Compatible with the SDIO card specification version 3.0.
- Designed to work with SD Memory, mini SD Memory, SDIO, mini SDIO, SD Combo, MMC, MMC plus, and MMC RS cards.
- Supports single block/multi-block read and write.
- Supports block sizes of 1 to 4096 bytes.
- Supports the write protection switch for write operations.
- Supports both synchronous and asynchronous abort.
- Supports auto CMD12 for multi-block transfer.
- Host can initiate non-data transfer command while data transfer is in progress.
- Allows cards to interrupt the host in 1-bit and 4-bit SDIO modes, also supports interrupt period.
- Embodies a fully configurable 128x32-bit FIFO for read/write data.
- Supports internal DMA capabilities.
- Supports advanced DMA to perform linked memory access.
- Supports little endian mode.

64.2 Features Not Supported

The model uSDHC does not support the following features:

- Pause during the data transfer at block gap not supported.
- SDIO read, wait and suspend resume operations not supported.
- External DMA capabilities not supported.
- Tuning not supported.
- Delay and clock related things not modeled.
- Non exact block read not supported.
- Frequency selection not supported.
- Speed modes not supported.
- Delay Line (DLL) read path not supported.
- Dual data rate mode not supported.
- Big endian mode not supported.

Chapter 65
Virtual Wrapper (Virt_Wrapper)

65.1 Introduction

Virtualization is implemented by protecting (for example, granting, or inhibiting) the access to a register within the virtualized module, dependent on the specific criteria. Virtualized module is the module instance associated with VIRT_WRAPPER. In this chip, SIUL2 is the virtualized module. Registers within virtualized module are virtualized by granting or inhibiting the access to different PDAC slots as encoded within virtualization pad assignments in VIRT_WRAPPER configuration registers. For this purpose, the virtualization information is programmed into VIRT_WRAPPER configuration registers that specify the grouping of registers within the virtualized module.

65.2 Supported Features

VIRT_WRAPPER supports the following features:

- A programmable chip-specific virtualization setup, capable to select a subset of the available initiators in combination with up to four available address spaces.
- Virtualizing accesses to the registers within a VM by inhibiting accesses to a subset of these registers under control of the specified virtualization information. Only write accesses can be inhibited for a specific initiator or set of initiators.

65.3 Assumptions

The following assumptions are taken while modeling this peripheral:

- All the registers can be configured with PDACs 0-15 (bitfield value can be 0-15) to group the corresponding pads.
- Since, VWRAP_SLOT_15 is assigned to the Model itself, selecting that would select the default PDAC_0.
- While trying to access PGPDO and MPGPDO registers, the check for missing pads are ignored.

Chapter 66
Virtualized Fault Collection and Control Unit (VFCCU)

66.1 Introduction

VFCCU collects faults and provides appropriate fault reactions which enables the device or software to take necessary action to move system to safe state.

66.2 Supported Features

The VFCCU block implements the following functionalities:

- Asynchronous fault capture from multiple sources.
- Hardware or software fault clearing management configuration on a per fault input basis.
- Reaction signalling for software reported faults.
- Per-fault DID configuration.
- DID and FHID based reaction signalling for each fault input.
- Multiple configurable timer registers for providing delayed reactions.
- Configurable per-DID based timer for delayed reactions.
- Freeze on first fault mode.

66.3 Features Not Supported

The VFCCU block does not implement the following functionalities:

- Async request-ack based handshake interface for fault signalling is not supported. All faults are triggered through rising or falling edge on p_fault_in input ports.
- Open-collector, open-drain, bistable, push-pull and fault-toggle mode not implemented.

66.4 Assumptions

The following assumptions are taken while modeling this peripheral:

- Faults configured as hardware-cleared are cleared when the corresponding fault input is de-asserted for rising-edge triggered faults or asserted for falling-edge triggered faults.

- DID for software triggered fault comes from AID of the transaction that writes the SWKSET register. Transaction AID is set by domain_id_extension.
- p_reaction_out[x] is asserted if any of the following are true:
  - A fault is being serviced by LVFCCU that has the corresponding RKNSEL bit in FHFLTRKC0_x register=0, and FHIMRKC0_0.RKNEN0=1.
  - The fault delayed reaction timer for a fault that has the corresponding RKNSEL bit in FHFLTRKC0_x register=0, and FHDLRKC0_0.RKNEN0=1.

66.5 SCML Commands

Name	Description
inject_fault	Inject hardware fault. Fault status bit is set regardless of edge polarity. Usage: inject_fault (fault_index, fault_did) • fault_index: Index of the HW fault to inject. Range is 0 - NUM_FAULT_INPUTS-1. • fault_did: Optional DID of injected fault. Range is 0-0xF. If not provided, the DID is taken from p_fault_did_in[fault_index], or the corresponding GFDID_Cx register.
clear_fault	Clear hardware fault. Fault status bit is cleared regardless of edge polarity or programmed software/hardware clearing. Usage: clear_fault(fault_index) fault_index: Index of the HW fault to clear. Range is 0 - NUM_FAULT_INPUTS-1.

Chapter 67
Wakeup Unit (WKPU)

67.1 Introduction

The Wakeup Unit (WKPU) supports 68 external sources that can generate interrupts or wakeup events and 6 external source(s) that can cause non-maskable interrupt request(s) or wakeup event(s). In addition, it combines its wakeup events with those generated from other wakeup sources to supply a single wakeup to the system.

67.2 Supported Features

The WKPU module implements the following features:

- All NMI configurations supported including.
- NMI destination source select.
- Selection of rising, falling, or both edges for interrupt.
- Capturing of second interrupt.
- Status and overrun flags.
- Asynchronous wakeup source.
- Up to 16 number of external wakeup sources or interrupts supporting.
- System interrupt vector for interrupt sources.
- Independent interrupt masks.
- Configurable system wakeup triggering from all interrupt sources.
- On chip wake support, mapped to same register as external wake/interrupt sources.

67.3 Features Not Supported

The following features are abstracted out or not implemented in WKPU module:

- Analog glitch filters are not modelled, although registers are provided.
- Pull up/down functionality is not modelled, although registers are provided.

Chapter 68
CANXL

68.1 Introduction

CANEXCEL is a communication controller implementing the CAN protocol according to the ISO118981:2015 standard and XL protocols based on CiA® 6101 version 1.0.0

68.2 Supported Features

The CANXL block implements the following functionalities:

- TX descriptor processing supported for FD, Classic, CANXL frames.
- RX descriptor processing supported for FD, Classic, CANXL frames.
- Configurable number of Transmit and Receive Message Descriptors.
- Transmit Message Descriptors can be configured to store multiple messages (Transmit Message Queues).
- Receive message descriptor can be configured to store multiple payloads (Receive Message Queues).
- Programmable transmission priority scheme: lowest ID, lowest descriptor number, or message local priority (defined by PRIO).
- Configurable number of retransmission attempts.
- Receive filter process for several message fields with mask and range selection.
- Filter Engine in RX processing supported in Bank0, Bank1, MDs, RXFIFO.
- Transmit and Receive Descriptor status (empty/full).
- ListenOnly mode, internal loopback mode supported.
- Host interrupts supported.
- Full 32bit address range supported for System Memory access.

68.3 Features Not Supported

The CANXL block does not implement the following functionalities:

- CRC computation and checking on CAN Bus not supported in CANXL.
- CAN HW bus errors like bit errors, stuffing error etc not supoported in CANXL.
- Data RAM ECC Memory Error not supported.
- Test mode access not supported in CANXL.

- Tx Timestamp value is not accurately modeled.
- Self Reception mode not supported.
- Transceiver Mode Switching not supported.
- Descriptor specific individual 1270 interrupt line not exposed from, instead ORed of those line is exposed.

68.4 Assumptions

None

68.5 SCML Commands

None

Chapter 69
NETC

69.1 Introduction

NETC is a TSN capable ethernet IP, which enables a comprehensive portfolio of ethernet solutions.

69.2 Supported Features

The NETC block implements the following functionalities:

- Enhance Configuration Access Mechanism (ECAM) supported.
- TxBD/RxBD ring supported.
- Ethernet MAC and Pseudo MAC supported.
- Parser functionality supported.
- VLAN classification supported.
- QoS classification supported.
- Ingress port filtering supported.
- Stream identification and filtering supported.
- Egress treatment supported.
- Ingress/Egress frame modification supported.
- Ingress VLAN processing supported.
- MAC learning and forwarding supported.
- IP multicast filtering supported.
- Egress VLAN processing supported.
- Ingress mirroring supported.
- L2 filtering supported.
- RFS classification supported.
- Soft/Warm and Function Level Reset (FLR) supported.
- VLAN tagged/untagged acceptance checking supported.
- Source MAC and VLAN antispoofing supported.
- VLAN insertion/removal supported.
- Transmit checksum offload supported.
- Following Table management commands supported:

  - MAC address filter table.
  - VLAN filter table.
  - RFS table.
  - Time gate scheduling table.
  - Ingress port filter table.
  - FDB table.
  - L2 IPv4 multicast filter table.
  - ETM class queue table.
  - ETM class scheduler table.
  - Ingress stream identification table
  - Ingress stream table
  - Ingress stream filter table
  - Egress treatment table
  - Stream gate instable table
  - Stream gate control list table
  - Ingress stream count table
  - Egress count table
  - Frame modification table
  - Frame modification data table
- Error codes from table management commands supported.
- PSIVSI messaging supported.
- Statistics counters (MAC/Port/SI) supported.
- MSIX supported.
- NXP switch tag supported.
- MACSEC Bypass mode, 128/256 bit encryption (including XPN mode) support added.

69.3 Features Not Supported

The NETC block does not implement the following functionalities:

- iEPRC INTx, power management and advanced error reporting not supported.
- IEEE 1588 timer functionality not supported.
- Trout module not supported.
- MDIO functionality not supported.
- Following Table management commands not supported:
  - Rate policer table.
  - Ingress sequence generation table.
  - Egress sequence recovery table.
  - Buffer pool table.
  - Shared buffer pool table.
  - ETM congestion group table.
  - ATS scheduler instance table.

  - ATS scheduler group instance table.
- Following error status not supported by Tx buffer writeback descriptor:
  - 0x002 Time specific departure drop.
  - 0x020 Frame dropped due to port reset.
  - 0x080 Frame too large for time gating window.
  - 0x0A0 Multibit ECC error.
  - 0x0F0 Parity error.
  - 0x100 Frame dropped due to switch congestion.
- RFS table statistics not supported.
- Seamless Redundancy (SR) sequence generation not supported.
- Frame replication and Elimination for Reliability (FRER) sequence generation/recovery (802.1CB) not supported.
- Egress queuing and traffic management not supported.
- Bridge forwarding storm control not supported.
- Signature duplicate filtering function not supported.
- MAC flow control (transmit pause and receive pause) not supported.
- MAC pre-emption (802.1Qbu) not supported.
- Cut through forwarding not supported.
- Credit based shaping (802.1 Qav) not supported.
- Time gating (802.Qbv) and Stream gating (802.1 Qci) functionality not supported.
- Time specific departure (TSD) not supported.
- Time capture not supported.
- Time stamp offload not supported.
- Port masquerading not supported.
- Rate policing not supported.
- Common internal memory/buffer management not supported.
- Congestion management not supported.
- Power management not supported.
- Error management not supported.
- Functional Safety not supported.
- MACSEC stats counter, error not supported.

69.4 Assumptions

None

69.5 SCML Commands

None

Chapter 70
NPU

70.1 Introduction

NPU is a neural processing unit designed to accelerate network model computations.

70.2 Supported Features

The NPU block implements the following functionalities:

- 1x Neutron accelerator model.
- RISCV core model.
- Data Mover DMA model.
- Weight decompressor model.
- MMR registers.
- 256KB SRAM TCM.
- Internal ITCM/DTCM.

70.3 Features Not Supported

The NPU block does not implement the following functionalities:

- Faults.
- Fault IRQ out port.
- Fault errors.
- ECC and Bus Errors.
- Performance counters.
- Event Trace.
- Listener Mode.

70.4 Assumptions

None

70.5 SCML Commands

None

Chapter 71
S32K5 HIERARCHICAL PATHS

71.1 Hierarchical Paths

S No.	Module name	Hierarchical Path
1	BCTU	/S32K56x_System/S32K56x_MCU/Analog/BCTU_0 /S32K56x_System/S32K56x_MCU/Analog/LPE_BCTU
2	CANXL	/S32K56x_System/S32K56x_MCU/Communication/CANXL
3	CMU_FC	/S32K56x_System/S32K56x_MCU/Safety/CMU_FC_[0-3] /S32K56x_System/S32K56x_MCU/Safety/CPE_CMU_FC_[0-10] /S32K56x_-System/S32K56x_MCU/Safety/LPE_CMU_FC_[0-5]
4	GPR_CPE	/S32K56x_System/S32K56x_MCU/Core_Systems/GPR_CPE
5	CRC	/S32K56x_System/S32K56x_MCU/Safety/CRC_[0-1] /S32K56x_Sys-tem/S32K56x_MCU/Safety/CPE_CRC_[0-1] /S32K56x_System/S32K56x_MCU/Safety/LPE_CRC
6	CSTCU_1	/S32K56x_System/S32K56x_MCU/Safety/CSTCU_1
7	eDMA5_CRC	/S32K56x_System/S32K56x_MCU/Safety/eDMA5_CRC_0
8	DMAMUX	/S32K56x_System/S32K56x_MCU/Core_Systems/DMAMUX_[0-1] /S32K56x_System/S32K56x_MCU/Core_Systems/DMAMUX_LPE
9	DSPI	/S32K56x_System/S32K56x_MCU/Communication/DSPI_MSC_[0-1]
10	DSPV	/S32K56x_System/S32K56x_MCU/DSPV
11	EIM	/S32K56x_System/S32K56x_MCU/Safety/EIM_[0-3] /S32K56x_System/S32K56x_MCU/Safety/CPE_EIM /S32K56x_Sys-tem/S32K56x_MCU/Safety/LPE_EIM
12	ERM	/S32K56x_System/S32K56x_MCU/Safety/ERM_[0-3] /S32K56x_System/S32K56x_MCU/Safety/LPE_ERM
13	ETH_PHY_-WRAPPER	/S32K56x_System/S32K56x_MCU/Peripheral_Stub/ETH_PHY_WRAPPER_stub
14	FLEXCAN	/S32K56x_System/S32K56x_MCU/Communication/FlexCAN_[0-16] /S32K56x_System/S32K56x_MCU/Communication/FlexCAN_LPE

15	FLEXIO	/S32K56x_System/S32K56x_MCU/Communication/FlexIO_[0-1]
16	FRO	/S32K56x_System/S32K56x_MCU/Clocking/FRO_1
17	FTM	/S32K56x_System/S32K56x_MCU/Timer/FTM_LPE
18	FXOSC	/S32K56x_System/S32K56x_MCU/Clocking/LPE_FXOSC
19	GPR_0	/S32K56x_System/S32K56x_MCU/Core_Systems/GPR_0_1
20	GPR_1	/S32K56x_System/S32K56x_MCU/Core_Systems/GPR_1_1
21	LPE_GPR_0	/S32K56x_System/S32K56x_MCU/Core_Systems/LPE_GPR0_1
22	LPE_GPR_1	/S32K56x_System/S32K56x_MCU/Core_Systems/LPE_GPR1_1
23	LCU	/S32K56x_System/S32K56x_MCU/Analog/LCU_[0-1] /S32K56x_System/S32K56x_MCU/Analog/LPE_LCU
24	LPCMP	/S32K56x_System/S32K56x_MCU/Analog/LPE_LPCMP_[0-2]
25	LPI2C	/S32K56x_System/S32K56x_MCU/Communication/LPI2C_[0-3] /S32K56x_System/S32K56x_MCU/Communication/LPE_LPI2C
26	LPSPI	/S32K56x_System/S32K56x_MCU/Communication/LPSPI_[0-5] /S32K56x_System/S32K56x_MCU/Communication/LPE_LPSPI_[0-1]
27	LPUART	/S32K56x_System/S32K56x_MCU/Communication/LPUART_[0-21] /S32K56x_System/S32K56x_MCU/Communication/LPUART_LPE_[0-2]
28	LSTCU	/S32K56x_System/S32K56x_MCU/Safety/LSTCU_1 /S32K56x_System/S32K56x_MCU/Safety/LSTCU_3 /S32K56x_System/S32K56x_MCU/Safety/LSTCU_4 /S32K56x_System/S32K56x_MCU/Safety/LSTCU_CPE /S32K56x_System/S32K56x_MCU/Safety/LSTCU_LPE /S32K56x_System/S32K56x_MCU/Safety/LSTCU_SOG /S32K56x_System/S32K56x_MCU/Safety/LSTCU_SYN_CORE /S32K56x_System/S32K56x_MCU/Safety/LSTCU_SYN_CORE_3 /S32K56x_System/S32K56x_MCU/Safety/LSTCU_SYN_CORE_12 /S32K56x_System/S32K56x_MCU/Safety/LSTCU_HSPI /S32K56x_System/S32K56x_MCU/Safety/LSTCU_NETC /S32K56x_System/S32K56x_MCU/Safety/LSTCU_SE
29	CGM	/S32K56x_System/S32K56x_MCU/Clocking/CGM_[0-5] /S32K56x_System/S32K56x_MCU/Clocking/CGM_LPE /S32K56x_System/S32K56x_MCU/Clocking/CGM_CPE
30	MC_ME	/S32K56x_System/S32K56x_MCU/Modes_power_Management/MC_ME_0 /S32K56x_System/S32K56x_MCU/Modes_power_Management/MC_ME_LPE /S32K56x_System/S32K56x_MCU/Modes_power_Management/MC_ME_CPE
31	MC_RGM	/S32K56x_System/S32K56x_MCU/Reset_Boot/MC_RGM_LPE

32	MCM	/S32K56x_System/S32K56x_MCU/Core_Systems/MCM_[0-3] /S32K56x_System/S32K56x_MCU/Core_Systems/MCM_LPE
33	MRAMC	/S32K56x_System/S32K56x_MCU/Memories/MRAMC_0
34	MRU	/S32K56x_System/S32K56x_MCU/Safety/MRU_[0-7] /S32K56x_System/S32K56x_MCU/Safety/MRU_CPE_[0-3] /S32K56x_System/S32K56x_MCU/Safety/MRU_LPE
35	MSCM	/S32K56x_System/S32K56x_MCU/Core_Systems/MSCM_0
36	MSGINTR	/S32K56x_System/S32K56x_MCU/Core_Systems/MSGINTR_[0-6]
37	NETC	/S32K56x_System/S32K56x_MCU/Communication/NETC_0
38	NPU	/S32K56x_System/S32K56x_MCU/NPU
39	MC_PCU	/S32K56x_System/S32K56x_MCU/Modes_power_Management/MC_PCU_0
40	PIT	/S32K56x_System/S32K56x_MCU/Timer/PIT_[0-2] /S32K56x_System/S32K56x_MCU/Timer/PIT_CPE_[0-1] /S32K56x_System/S32K56x_MCU/Timer/PIT_LPE
41	PLL	/S32K56x_System/S32K56x_MCU/Clocking/PLL_[0-1] /S32K56x_System/S32K56x_MCU/Clocking/PLL_CPE
42	PMC	/S32K56x_System/S32K56x_MCU/Modes_power_Management/LPE_PMC
43	POR_GEN	/S32K56x_System/S32K56x_MCU/Reset_Boot/POR_GEN_LPE
44	RDC	/S32K56x_System/S32K56x_MCU/Reset_Boot/RDC_[0-1]
45	RTC	/S32K56x_System/S32K56x_MCU/Timer/RTC_LPE
46	RXLUT	/S32K56x_System/S32K56x_MCU/Communication/RXLUT_0
47	SAI	/S32K56x_System/S32K56x_MCU/Communication/SAI_[0-1]
48	SARADC	/S32K56x_System/S32K56x_MCU/Analog/SARADC_[0-1] /S32K56x_System/S32K56x_MCU/Analog/LPE_SARADC
49	SEMA4	/S32K56x_System/S32K56x_MCU/Core_Systems/SEMA4_[0-1] /S32K56x_System/S32K56x_MCU/Core_Systems/SEMA4_LPE /S32K56x_System/S32K56x_MCU/Core_Systems/SEMA4_CPE
50	SINC	/S32K56x_System/S32K56x_MCU/Analog/SINC_0
51	SIUL2	/S32K56x_System/S32K56x_MCU/Core_Systems/SIUL2_[0-4] /S32K56x_System/S32K56x_MCU/Core_Systems/SIUL2_LPE
52	SRAMC	/S32K56x_System/S32K56x_MCU/Memories/SRAMCTL_[0-2]
53	STM	/S32K56x_System/S32K56x_MCU/Timer/Stm_[0-3] /S32K56x_System/S32K56x_MCU/Timer/Stm_CPE_[0-1] /S32K56x_System/S32K56x_MCU/Timer/Stm_LPE

54	SWT	/S32K56x_System/S32K56x_MCU/Timer/SWT_[0-3] /S32K56x_System/S32K56x_MCU/Timer/SWT_CPE_[0-1] /S32K56x_System/S32K56x_MCU/Timer/SWT_STARTUP /S32K56x_System/S32K56x_MCU/Timer/SWT_LPE
55	SXOSC	/S32K56x_System/S32K56x_MCU/Clocking/LPE_SXOSC
56	TEMPSENSE	/S32K56x_System/S32K56x_MCU/Analog/LPE_TEMPSENSE
57	TRGMUX	/S32K56x_System/S32K56x_MCU/Analog/TRGMUX_0 /S32K56x_System/S32K56x_MCU/Analog/TRGMUX_LPE /S32K56x_System/S32K56x_MCU/Analog/TRGMUX_MSC
58	uSDHC	/S32K56x_System/S32K56x_MCU/Communication/uSDHC/uSDHC_0
59	VFCCU	/S32K56x_System/S32K56x_MCU/Safety/VFCCU_[0-3] /S32K56x_System/S32K56x_MCU/Safety/LPE_VFCCU /S32K56x_System/S32K56x_MCU/Safety/CPE_VFCCU
60	VIRT_- WRAPPER	/S32K56x_System/S32K56x_MCU/Core_Systems/VWRAP_[0-4] /S32K56x_System/S32K56x_MCU/Core_Systems/LPE_VWRAP
61	WKPU	/S32K56x_System/S32K56x_MCU/Modes_power_Management/WKPU_LPE
62	XBIC	/S32K56x_System/S32K56x_MCU/Core_Systems/XBIC_AUX /S32K56x_System/S32K56x_MCU/Core_Systems/XBIC_PER /S32K56x_System/S32K56x_MCU/Core_Systems/XBIC_TCM /S32K56x_System/S32K56x_MCU/Core_Systems/LPE_XBIC
63	XRDC	/S32K56x_System/S32K56x_MCU/Core_Systems/XRDC_MRC_PDAC_CPE /S32K56x_System/S32K56x_MCU/Core_Systems/XRDC_MRC_PDAC_LPE /S32K56x_System/S32K56x_MCU/Core_Systems/XRDC_DAC_LPE /S32K56x_System/S32K56x_MCU/Core_Systems/XRDC_DAC_CPE /S32K56x_System/S32K56x_MCU/Core_Systems/XRDC_DAC_A /S32K56x_System/S32K56x_MCU/Core_Systems/XRDC_DAC_C /S32K56x_System/S32K56x_MCU/Core_Systems/XRDC_DAC_B /S32K56x_System/S32K56x_MCU/Core_Systems/XRDC_MRC_PDAC_A /S32K56x_System/S32K56x_MCU/Core_Systems/XRDC_MRC_PDAC_C /S32K56x_System/S32K56x_MCU/Core_Systems/XRDC_MRC_PDAC_B
64	XSPI	/S32K56x_System/S32K56x_MCU/Communication/XSPI_0
65	eDMA3	/S32K56x_System/S32K56x_MCU/Core_Systems/eDMA3_LPE
66	eDMA5	/S32K56x_System/S32K56x_MCU/Core_Systems/eDMA5_0
67	INTC	/S32K56x_System/S32K56x_MCU/DSPV/INTC_1_0 /S32K56x_System/S32K56x_MCU/HSE2/HSE2_SS/Core_Modules/INTC_0
68	MU	/S32K56x_System/S32K56x_MCU/DSPV/MU[0-3] /S32K56x_System/S32K56x_MCU/HSE2/HSE2_SS/Security_Modules/SENT_- MU_[0-7]

69	SECURE_-GPR	/S32K56x_System/S32K56x_MCU/HSE2/HSE2_SS/External_Func_Modules/SECURE_GPR_1_[0-1] /S32K56x_System/S32K56x_MCU/HSE2/HSE2_SS/External_Func_Modules/LPE_SECURE_GPR_1_0